



Aristotle University of Thessaloniki
Department of Physics



Design and Implementation of Spatial Image Filters on FPGA

Nefeli Zikou
AEM: 15402

Supervising Professor: Konstantinos Siozios

Thessaloniki, September 2026



ΑΡΙΣΤΟΤΕΛΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΟΝΙΚΗΣ
ΤΜΗΜΑ ΦΥΣΙΚΗΣ



Σχεδίαση και Υλοποίηση Αλγορίθμων Επεξεργασίας Εικόνας σε Διατάξεις FPGA

Νεφέλη Ζήκου

AEM: 15402

Επιβλέπων Καθηγητής: Κωνσταντίνος Σιώζιος

Θεσσαλονίκη, Σεπτέμβριος 2026

ABSTRACT

Spatial filtering is a core operation in image processing, that requires multiple operations per pixel resulting in high computational cost. FPGAs are a common hardware acceleration platform used to tackle this problem. The choice of numeric representation and architecture influences the resource cost, the operating speed and the output accuracy of such designs.

This thesis presents a controlled comparison of two arithmetic implementations of 2D spatial filters on an FPGA. Sobel, Gaussian and cascaded Gaussian-Sobel filters were implemented in SystemVerilog for 3x3 and 5x5 kernels, each as a multiplier-less integer (shift-and-add) datapath and as a custom IEEE 754 single-precision floating-point datapath, resulting in twelve designs on an AMD Artix-7 device. The target device, the hardware description language, the test image, the streaming architecture and the evaluation scripts are kept fixed, while the arithmetic, the kernel size and the filter are varied.

Each design was evaluated with post-implementation resource and timing reports and with the image quality metrics: Max Absolute Difference, Mean Squared Error, Root Mean Squared Error, Peak Signal-to-Noise Ratio and Structural Similarity Index Measure, against a reference of the same filter created with Python.

The results show that integer arithmetic is preferable for filters with coefficients that are integer values or can be approximated with an acceptable error. The additional precision of floating-point datapaths is useful for fractional coefficients and multistage pipelines, where quantization error accumulates between the stages and affects the result. In general, integer arithmetic uses significantly less resources than the floating-point equivalent. The findings are limited to one test image, and the performance metrics are derived from simulation and implementation reports, not from a test on a physical FPGA board.

Keywords: FPGA, spatial filtering, Sobel filter, Gaussian filter, fixed-point arithmetic, floating-point arithmetic, SystemVerilog, image quality, hardware resources

ΠΕΡΙΛΗΨΗ

Το χωρικό φιλτράρισμα (spatial filtering) είναι μια βασική λειτουργία στην επεξεργασία εικόνας, η οποία απαιτεί πολλαπλές πράξεις ανά εικονοστοιχείο και οδηγεί σε υψηλό υπολογιστικό κόστος. Οι διατάξεις FPGA (Συστοιχία Επιτόπια Προγραμματιζόμενων Πυλών) αποτελούν μια συνηθισμένη πλατφόρμα επιτάχυνσης υλικού για την αντιμετώπιση αυτού του προβλήματος. Η επιλογή της αριθμητικής αναπαράστασης και της αρχιτεκτονικής επηρεάζει το κόστος σε πόρους, την ταχύτητα λειτουργίας και την ακρίβεια της εξόδου τέτοιων διατάξεων.

Η παρούσα διπλωματική εργασία παρουσιάζει μια ελεγχόμενη σύγκριση δύο αριθμητικών υλοποιήσεων δισδιάστατων χωρικών φίλτρων σε FPGA. Τα φίλτρα Sobel, Gauss και διαδοχικά (cascaded) φίλτρα Gauss-Sobel υλοποιήθηκαν σε SystemVerilog για μάσκες μεγέθους 3x3 και 5x5, με ακέραια αριθμητικής χωρίς πολλαπλασιαστές (shift-and-add) και με αριθμητική κινητής υποδιαστολής κατά το πρότυπο IEEE 754. Συνεπώς προέκυψαν δώδεκα υλοποιήσεις στην πλακέτα AMD Artix-7. Η πλακέτα, η γλώσσα περιγραφής υλικού, η εικόνα δοκιμής, η αρχιτεκτονική ροής (streaming) και ο τρόπος αξιολόγησης των αποτελεσμάτων διατηρούνται σταθερά, ενώ μεταβάλλονται η αριθμητική, το μέγεθος του πυρήνα και το φίλτρο.

Κάθε υλοποίηση αξιολογήθηκε χρησιμοποιώντας αναφορές πόρων και χρονισμού, καθώς και με τους δείκτες ποιότητας εικόνας: Μέγιστη Απόλυτη Διαφορά (Max Absolute Difference), Μέσο Τετραγωνικό Σφάλμα (Mean Squared Error), Ρίζα Μέσου Τετραγωνικού Σφάλματος (Root Mean Squared Error), Λόγος Σήματος προς Θόρυβο Κορυφής (Peak Signal-to-Noise Ratio) και Δείκτης Δομικής Ομοιότητας (Structural Similarity Index Measure), ως προς μια εικόνα αναφοράς με το ίδιο φίλτρο που δημιουργήθηκε σε Python.

Τα αποτελέσματα δείχνουν ότι η ακέραια αριθμητική είναι προτιμότερη για φίλτρα με συντελεστές που είναι ακέραιες τιμές ή μπορούν να προσεγγιστούν με αποδεκτό σφάλμα. Η επιπλέον ακρίβεια που προσφέρει η αριθμητική κινητής υποδιαστολής είναι χρήσιμη για κλασματικούς συντελεστές και για υλοποιήσεις με πολλαπλά στάδια, όπου το σφάλμα συσσωρεύεται μεταξύ των σταδίων και επηρεάζει το αποτέλεσμα. Ακόμη, η ακέραια αριθμητική χρησιμοποιεί σημαντικά λιγότερους πόρους από την αντίστοιχη υλοποίηση με κινητή υποδιαστολή. Τα πορίσματα περιορίζονται σε μία δοκιμασμένη εικόνα, και οι δείκτες επίδοσης προκύπτουν από αναφορές προσομοίωσης και υλοποίησης, όχι από δοκιμή σε πραγματική διάταξη FPGA.

Λέξεις-κλειδιά: FPGA, χωρικά φίλτρα, φίλτρο Sobel, φίλτρο Gaussian, αριθμητική σταθερής υποδιαστολής, αριθμητική κινητής υποδιαστολής, SystemVerilog, ποιότητα εικόνας, πόροι υλικού

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to my supervisor, Konstantinos Siozios, for giving me the opportunity to work on this thesis. His guidance, continuous support and patience were instrumental in the completion of this work.

I am also deeply grateful to my family and friends, for their unwavering encouragement during this journey.

Finally, I would like to thank my cat for his moral support and modeling contributions.

CONTENTS

CHAPTER 1:	Introduction.....	5
1.1.	Introduction	5
1.2.	Thesis Objectives and Research Questions.....	5
CHAPTER 2:	Background.....	6
2.1.	Architectures.....	6
2.1.1.	CPUs, GPUs and FPGAs.....	6
2.1.2.	FPGA Architecture and Hardware Components.....	6
2.1.3.	Basic Design.....	8
2.1.4.	Line Buffer Architecture	9
2.2.	Number Formats & Hardware Arithmetic	10
2.2.1.	Data Representations.....	10
2.2.2.	Arithmetic Architectures	11
2.3.	Spatial Filtering.....	13
2.3.1.	Spatial Convolution	13
2.3.2.	Selected Filters	14
2.4.	Tools & Development Methodology.....	17
2.4.1.	Hardware Description Methods.....	17
2.4.2.	Xilinx Vivado Workflow	17
2.5.	Comparison & Evaluation Metrics	19
2.5.1.	Image Quality and Similarity	19
2.5.2.	Hardware & Performance Metrics	20
CHAPTER 3:	Literature Review	22
3.1.	Hardware architecture for spatial convolution.....	22
3.2.	FPGA implementations for spatial filtering.....	23
3.3.	Fixed-point and floating-point arithmetic.....	24
3.4.	Kernel size scaling and cascaded filter errors	24
3.5.	Motivation for this thesis	25
CHAPTER 4:	Methodology	26
4.1.	Preprocessing.....	26
4.1.1.	Preparation of input file	26
4.2.	Filter execution in hardware	27
4.2.1.	Line Buffer Module.....	27
4.2.2.	Top Module	29
4.2.3.	Integer Implementation	29

4.2.4.	IEEE 754 Floating-point Implementation	31
4.2.5.	Cascaded Filters.....	33
4.2.6.	Testbench	34
4.3.	Post-processing	34
4.3.1.	Custom Filters.....	34
4.3.2.	Image Reconstruction	35
4.3.3.	Image Comparison.....	35
4.3.4.	Hardware Performance and Resource Metrics	35
CHAPTER 5:	Results	36
5.1.	Preprocessing	36
5.2.	Hardware Synthesis and Implementation	37
5.2.1.	Hardware Resource Metrics	38
5.2.2.	Timing Analysis, Execution Cycles and FPS.....	41
5.3.	Post processing and Quality verification.....	42
5.3.1.	Software Baseline Verification	42
5.3.2.	Hardware Image Reconstruction.....	43
5.3.3.	Image Comparison.....	44
CHAPTER 6:	Discussion	48
6.1.	Major findings	48
6.1.1.	Hardware Resource Metrics and Timing Analysis	48
6.1.2.	Image Quality	48
6.1.3.	Quality compared to resource cost	49
6.2.	Summary of findings and answers to research questions	50
6.3.	Limitations.....	51
6.4.	Future work.....	51
6.5.	Conclusions	52
References		53

List of Figures

Figure 2-1: Schematic representation of basic FPGA architecture.	7
Figure 2-2: Schematic representation of square-wave clock signal.	8
Figure 2-3: Schematic representation of Line Buffer architecture with two lines.	9
Figure 2-4: Representation of number -123.42 in IEEE 754 single-precision floating-point format.	11
Figure 2-5: Example of bit-shifts used for multiplication and division with powers of two.	12
Figure 2-6: Schematic of image convolution with a 3x3 kernel.	13
Figure 2-7: Screenshot of the signal waveforms from Vivado Behavioral simulation.	18
Figure 2-8: Post-implementation device view and zoomed in details, showing placed and routed slices on clock regions of the target FPGA.	18
Figure 4-1: Overall workflow, including inputs and outputs.	26
Figure 4-2: Schematic of an original ($W \times H$) and a padded image $W + 4 \times H + 4$, for cascaded filters with $K=3 \Rightarrow N=1$ or a stand-alone with $K=5 \Rightarrow N=2$	26
Figure 4-3: Process of converting original image to binary text file.	27
Figure 4-4: Representation of convolution kernel inside and out of the image limits and coordinate tracked pixel.	28
Figure 4-5: Architecture of the Top module for a stand-alone filter.	29
Figure 4-6: Architecture of the Top module for cascaded filters.	29
Figure 4-7: Architecture of the floating-point implementation.	31
Figure 4-8: Adder tree stages and propagation of valid signal for the 3x3 Gauss core filter module.	33
Figure 5-1: Schematic representation of the image pixel values converted to a sequence of binary values in the text file.	36
Figure 5-2: The original RGB image (left) and the 8-pixel padded grayscale transformation (right).	37
Figure 5-3: Percentage of total board hardware resources used by kernel size and architecture.	38
Figure 5-4: Post-implementation device view of Sobel filter architectures.	39
Figure 5-5: Post-implementation device view of Gauss filter architectures.	39
Figure 5-6: Post-implementation device view of Cascaded filters architectures.	39
Figure 5-7: Hardware area efficiency calculated as achieved throughput per kLUT used, for all cases.	40
Figure 5-8: Total LUTs used and the respective RMSE achieved for each case.	40
Figure 5-9: Maximum Operating Frequency and Worst Negative Slack for each case.	41
Figure 5-10: Total operating clock cycles and initial latency for each filter architecture.	41
Figure 5-11: Achieved Frame Rate for all cases.	42
Figure 5-12: Absolute difference heat map and SSIM plot of custom 3x3 Sobel filter implementation and OpenCV equivalent.	42
Figure 5-13: Comparison of custom 3x3 Sobel filter (left) and OpenCV equivalent (right).	43
Figure 5-14: Reconstructed images for Sobel filter.	43
Figure 5-15: Reconstructed images for Gaussian filter.	44
Figure 5-16: Reconstructed images for cascaded filters.	44
Figure 5-17: Absolute difference heat map & SSIM plot of all Sobel filter cases and respective reference. ...	45
Figure 5-18: Absolute difference heat map of integer & floating-point 3x3 Gaussian filter implementation.	46
Figure 5-19: Absolute difference heat map of integer & floating-point 5x5 Gaussian filter implementation ..	46
Figure 5-20: Absolute difference heat map of integer & floating-point 3x3 cascaded filters implementation	47
Figure 5-21: Absolute difference heat map of integer & floating-point 5x5 cascaded filters implementation	47

List of Tables

Table 2-1: Exact coefficient values for the Gaussian filter kernels.....	16
Table 4-1: Pixel value transformations for a 3x3 line buffer.	28
Table 4-2: Final gradient calculation using bit-shifts for Sobel and Gaussian filter, kernel sizes 3x3 and 5x5.	30
Table 5-1: Dimensions of padded images for each filter case, with initial image size (364x483).....	37
Table 5-2: Hardware usage and timing metrics for all filter architectures.	37
Table 5-3: Calculated usage and timing metrics for all filter architectures.	38
Table 5-4: Image quality metrics for custom filter implementation in Python.....	43
Table 5-5: Image comparison metrics for all filter cases with 3x3 window.	44
Table 5-6: Image comparison metrics for all filter cases with 5x5 window.	45

CHAPTER 1: Introduction

1.1. Introduction

Spatial filtering is one of the most common operations in digital image processing. Filters such as Sobel and Gaussian are used for edge detection, smoothing and noise reduction, and they are the basis of many computer vision pipelines. Field Programmable Gate Arrays (FPGAs) are suitable for these applications because they can execute heavy calculations at an accelerated rate, providing a continuous pixel stream.

Their performance depends on design choices like the numeric representation of the data, the arithmetic architecture and the size of the kernel. All of those affect the number of resources used, the maximum operating frequency and the accuracy of the result. For example, floating-point arithmetic offers high precision at the cost of more hardware, while integer and shift-and-add arithmetic is cheaper but usually more prone to rounding errors.

This thesis develops SystemVerilog FPGA architectures for 3×3 and 5×5 Sobel, Gaussian and cascaded Gaussian-Sobel filters. Each filter is implemented in two ways, as a multiplier-less integer (shift-and-add) datapath and as a custom IEEE 754 single-precision floating-point datapath, and all designs target an AMD Artix-7 FPGA. Twelve implementations are evaluated using post-implementation resource and timing reports, behavioral-simulation, the reconstructed output images, and image-quality metrics.

The target is to create a controlled comparison. The target device, the hardware description language, the test image, the streaming framework (line buffer, sliding window and control logic) and the evaluation scripts are kept fixed, while the arithmetic representation, the architecture and the kernel size are varied to understand the effect of each choice.

1.2. Thesis Objectives and Research Questions

The main goal of this thesis is to study how hardware resource usage and output accuracy change when implementing spatial image processing filters using different techniques on an FPGA.

In particular, the work focuses on the following research questions:

RQ1. How does changing from floating-point arithmetic to integer fixed-point approximations affect image quality compared to savings in hardware resource usage?

RQ2. How do hardware resource consumption, maximum operating frequency ($F_{\max}$), and latency change when the kernel size increases from 3x3 to 5x5?

RQ3. How much do errors accumulate when connecting multiple filters in a sequence using fixed-point arithmetic, compared to a floating-point implementation?

RQ4. How do the achievable frame rate and the hardware area efficiency differ between the integer and floating-point datapaths, and how do they scale with kernel size?

CHAPTER 2: Background

2.1. Architectures

2.1.1. CPUs, GPUs and FPGAs

Central Processing Unit (CPU) is the primary processing unit found inside common computers. It consists mainly of the Control Unit and the Arithmetic Logic Unit (ALU). During each cycle the CPU fetches, decodes and executes several instructions. CPUs support many programming frameworks, making them relatively easy to program. However, because they process information sequentially, they are less efficient for large-scale computations [1]. Modern CPUs have a small number of powerful cores and can perform image-processing tasks using features such as Single Instruction, Multiple Data (SIMD) and cache memory [2].

Graphics Processing Unit (GPU). The primary function of a GPU is the rapid execution of graphics-related tasks. GPUs contain thousands of small, relatively simple cores that can perform many computations concurrently, making them suitable for use as general-purpose processing accelerators [1]. GPU programming frameworks and languages, such as CUDA and OpenCL, enable tasks to be decomposed and executed using the Single Instruction, Multiple Threads (SIMT) model. In this model, a single instruction is applied concurrently across multiple threads, while the hardware schedules and switches between groups of threads [3]. In image-processing applications, GPUs can provide high computational throughput; however, data transfer between processing units and memory may introduce latency and increase power consumption [4].

Field Programmable Gate Arrays (FPGAs) are general purpose logic devices that can be reconfigured after manufacturing. They consist of programmable logic blocks and interconnects that can be configured to form circuits implementing the desired operation directly. Unlike a conventional processor, an FPGA does not have a fixed datapath, so the operation is implemented as dedicated hardware and not as a sequence of software instructions. This flexibility makes FPGAs suitable for applications such as parallel processing, artificial intelligence, networking, and image processing. The cost of this flexibility is that programmable logic is slower and occupies a larger area than fixed circuitry [5], [6]. FPGA designs use parallelism and pipelining to achieve high performance. Their development also typically requires greater effort because circuits are specified at the register-transfer level (RTL) using hardware description languages (HDLs), rather than conventional programming languages.

A comparative study showed that no single platform performs best in all cases. A CPU, GPU and FPGA were evaluated using vendor-optimized libraries to execute computer-vision kernels. The GPU was more energy-efficient for simple kernels, but the FPGA performed better for more complex kernels and complete processing pipelines, with the advantage increasing with pipeline complexity [2]. This advantage is generally attributed to the ability of FPGAs to keep data on chip, so repeated external memory access is reduced.

2.1.2. FPGA Architecture and Hardware Components

As described in Figure 2-1, at an architectural level, an FPGA is built from [7], [8]:

1. Logic Blocks for combinational and sequential logic.
2. Dedicated arithmetic units (DSP blocks).
3. On-chip memory blocks.
4. Input/Output interfaces.
5. Clock Infrastructure, for maintaining synchronization.
6. Programmable Interconnect, for routing connections between resources.

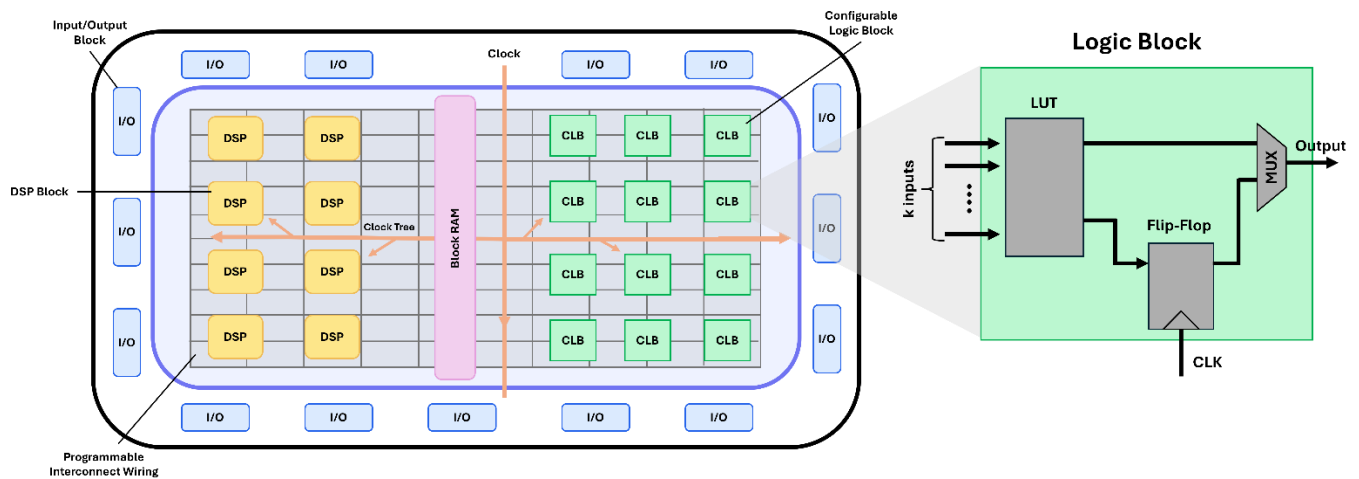


Figure 2-1: Schematic representation of basic FPGA architecture.

The basic logic element in most FPGAs consists of a **Look-Up Table (LUT)** that executes combinational logic and a **Flip-Flop (FF/register)**, a 1-bit storage element. A k -input LUT stores a precalculated truth table of a Boolean function, capable of implementing arbitrary combinational logic with up to k input variables. The register stores the LUT output on a clock edge, enabling multi-stage pipelined and synchronous designs. In addition to general combinational logic, LUTs can be grouped into larger logic blocks and configured as small memories (distributed RAM) or shift structures (Shift-Register LUTs), which are useful for local buffering.

Digital Signal Processing Blocks (DSPs) are hardened arithmetic units designed for high-rate numeric computation. A typical DSP block supports operations such as multiply, add and multiply-accumulate (MAC), which are executed in parallel. Using DSP blocks instead of LUT-based multipliers improves speed, reduces logic utilization and raises the achievable maximum clock frequency ($F_{\max}$).

Block RAM (BRAM) is on-chip storage for large data sets built from dedicated static RAM blocks. It is used in image processing pipelines, FIFO queues and line buffers. In 2D spatial convolution architectures, BRAMs buffer full image lines locally, and give kernels access to neighborhood pixels simultaneously, allowing the design to produce one pixel per cycle. FPGAs also provide Distributed RAMs, small memories built from LUTs that, unlike BRAM, are spread across the device. Their flexible placement makes them easily available close to compute logic. Another option is off-chip memory interfaces that are used when the data exceed on-chip capacity.

Input/Output (I/O) blocks allow the FPGA to connect with cameras, displays, memory chips, and other external devices. They provide programmable interfaces that support various signal types, while reducing wiring complexity.

Clock Infrastructure drives the design and sets its speed and data processing rate. At least one clock is used in each design, and the clock signal is distributed to all elements on the chip, with minimal timing variation. The infrastructure ensures that clock edges arrive simultaneously to FFs and DSPs to produce a synchronized response.

The **programmable interconnect** is the physical mesh of wires and programmable switch matrices that performs the routing between resources. It connects logic, DSP, BRAM, and I/O elements. While it provides architectural flexibility, signal delays in the interconnect lines can negatively affect slack and operational timing limits.

2.1.3. Basic Design

FPGA designs are usually synchronous, meaning that state changes are synchronized to a single global square wave **Clock Signal (clk)**. The period of this clock (T_{clk}) is the duration of one full cycle that consists of two half phases: high and low, and the transitions between them are defined as positive and negative edges. Figure 2-2 shows a representation of this signal. Clock edges arrive simultaneously across the architecture, so the signal activates synchronized state transitions in all elements. For example, FFs are usually triggered by a positive clock edge, so the output updates only when the signal changes from a low to a high (low-to-high transition), and not in between cycles [9].

The **Clock Frequency (F)**, the number of clock cycles completed per second, is calculated as the inverse of the clock period [2-1]. For example, a clock period of $T_{clk} = 20\text{ns}$ results in a frequency of $F = 50\text{MHz}$.

$$F = \frac{1}{T_{clk}} \quad [2-1]$$

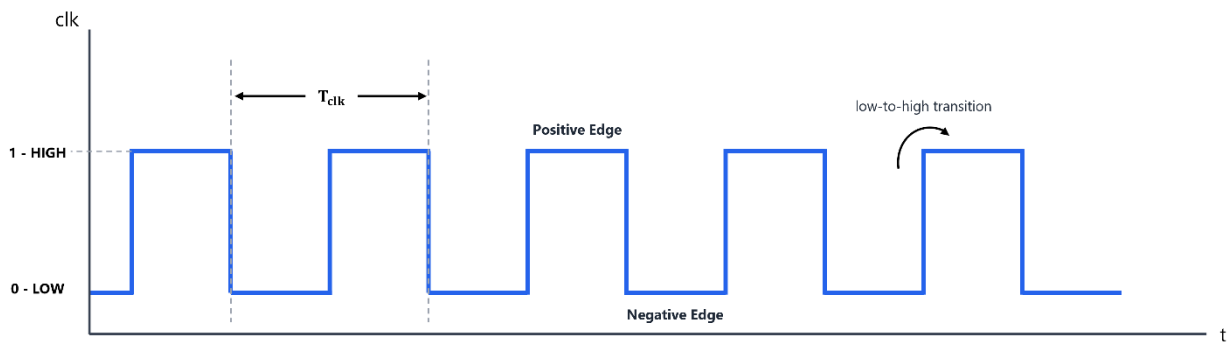


Figure 2-2: Schematic representation of square-wave clock signal.

Other ways to ensure flow control, alongside clock signals, is a stage-by-stage “handshake” chain. After one stage has created data, the following stage may not be ready to accept them. For the data transfer to happen, the source produces a valid signal to send data, and the receiver produces a ready signal when it can accept them, with the transfer happening on a positive clock edge.

A timing path begins at a source (launching) register, goes through combinational gates and routing, and ends at a destination (capturing) register. For a circuit to operate reliably (without timing violations), data from the source register must arrive at the destination before the next clock edge. Slack is the difference between the time at which data should arrive and the time at which it actually arrives, and it is an indicator of timing closure and operational reliability of the design. The longest (slowest) combinational logic path in a circuit, called the **Critical Path**, defines the **Worst Negative Slack (WNS)**, the smallest slack across all timing paths. WNS allows for the optimization of a circuit through the calculation of the **Minimum Operating Period (T_{min})** and **Maximum Operating Frequency (F_{max})**. Positive WNS ($WNS \geq 0$) values show that there is margin for decreasing clock period and increasing frequency, directly improving performance [10].

$$T_{min} = T_{target} - WNS \quad [2-2]$$

$$F_{max} = \frac{1}{T_{target} - WNS} \quad [2-3]$$

F_{max} can also be increased by splitting long paths with register stages. This process is called pipelining. Each insertion shortens the longest combinational path, so it improves WNS, at the cost of one extra cycle of delay.

Pipelining will also increase **Latency (L)**, which is the duration in clock cycles, between the first input event and the corresponding output. It shows the depth of the design and serves as a metric of how quickly it produces results. A multiplier-less integer shift-and-add architecture could complete in a single cycle ($L = 1$), compared to multi-stage floating-point pipelines that usually require multiple cycles. In a filter architecture that utilizes line buffers, the total latency includes the cycles needed to fill the line buffers (L_{warmup}) and the initial depth of the design [11].

The number of cycles that need to pass between two consecutive inputs is defined as **Initiation Interval (II)**. A fully pipelined filter with no delays has $II = 1$, meaning it can process one input pixel and generate one output pixel per cycle, after the initial latency. **Throughput (T)**, the rate at which data is processed by the design, is inversely correlated with II , as is **Frame Rate (FPS)**. For an image with dimensions ($W \times H$), where pixels are processed at $II = 1$, the number of cycles required to process the full frame equals the number of pixels streamed into the design, plus the pipeline depth D , that is the extra cycles needed to flush the last pixel through the datapath, so FPS and T are calculated as:

$$\text{FPS} = \frac{F_{\text{max}}}{\text{Total Cycles per Frame}} = \frac{F_{\text{max}}}{(W \cdot H) + D} \quad [2-4]$$

$$T = \frac{F_{\text{max}}}{II} \quad [2-5]$$

In this thesis the testbench streams pixels and accepts outputs continuously, so the pipeline never stalls and the total cycle count equals the padded image size (the input) and the pipeline depth.

2.1.4. Line Buffer Architecture

Image analysis applications that use spatial convolution require the neighborhood around each target pixel for the convolution calculation. Linear streamed input does not provide access to future rows and columns needed for the calculation, and full frame buffering stores the entire 2D image matrix, usually off-chip, increasing resource costs and delays.

A Line Buffer is a memory structure that stores previous image rows, and combined with Shift Registers, it generates a sliding window with all neighborhood pixels in every cycle. For a $K \times K$ window, $K - 1$ full image lines are stored and combined with the following row they form the full neighborhood matrix [12] (Figure 2-3).

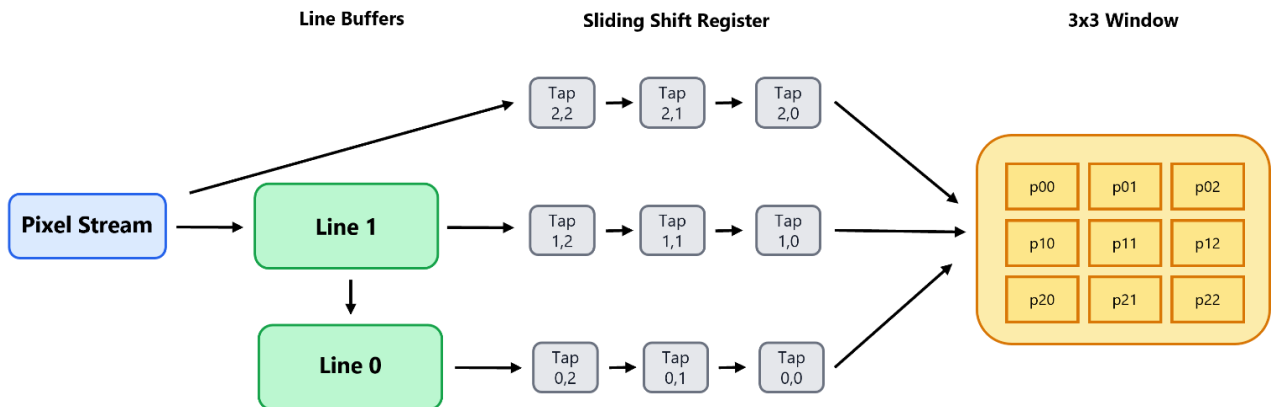


Figure 2-3: Schematic representation of Line Buffer architecture with two lines.

The structure produces a valid window after the warm-up latency has passed, meaning after the window is filled with pixel values. For a design that produces one pixel per cycle, the line buffer latency in cycles is calculated in [2-6].

$$L_{\text{warmup}} = (K - 1) \cdot W + K \quad [2-6]$$

where W is the width of the input image.

Errors can occur when the window approaches the edges of the image, because pixels of the right side of *line_y* will be mixed with the left side of *line_{y+1}*. Boundary handling techniques prevent that, for example image padding and tracking window coordinates.

2.2. Number Formats & Hardware Arithmetic

2.2.1. Data Representations

The selection of numeric representations is an important architectural decision for image processing in FPGA. It directly affects resource utilization, maximum operating frequency, latency, and numerical correctness. The main representations are analyzed below [13], [14].

Fixed-point Representation

Fixed-point arithmetic represents fractional numbers using a constant number of bits allocated to the integer and fractional parts. The common notation is *Qm.f*, where *m* is the number of integer bits and *f* the number of fractional bits, a total length of *m + f* bits with an additional one for signed values.

A fixed-point number is stored in hardware as a plain integer, and the position of the binary point is fixed by the format. A real value *V* is stored as the integer *X*, and the conversion between them is described in [2-7].

$$\begin{aligned} X &= \text{round}(V \cdot 2^f) \\ V &= X \cdot 2^{-f} \end{aligned} \quad [2-7]$$

where *V* is the real value, *X* the stored integer and *f* the number of fractional bits.

Fixed-point is usually the preferred representation because the width of the integer and fractional parts can be changed per stage. This allows efficient use of hardware resources while maintaining accuracy required at each stage.

Integer Representation

Integers are a special case of fixed-point numbers where the fractional part is zero (*f = 0*), so the binary point is implicitly positioned after the least significant bit. Unsigned integer formats are for non-negative values, and signed values are represented using two's-complement. For an *N*-bit format, the value ranges are given by [2-8]. Integer arithmetic has the lowest implementation complexity and is usually used in the first and last stages of image pipelines where pixel values are in the range of [0, 255], so they can be fully described with 8-bit unsigned integers.

$$\begin{aligned} \text{Unsigned range: } & [0, 2^N - 1] \\ \text{Signed range: } & [-2^{N-1}, 2^{N-1} - 1] \end{aligned} \quad [2-8]$$

Integer arithmetic provides hardware efficiency, since calculations are inexpensive and can be performed in a single cycle. The obvious limitation is the absence of fractional precision and the potential for overflow.

IEEE 754 Single-Precision Floating-Point Representation

IEEE 754 single-precision floating-point uses 32 bits composed of a 1-bit sign (S), an 8-bit biased exponent (E), and a 23-bit mantissa or fractional (M). It is expressed as:

$$A = (-1)^S \cdot (1.M) \cdot 2^{E-127} \quad [2-9]$$

The structure of the number starts with the sign at bit 31, where a single bit indicates positive (0) or negative (1) sign. Bits 30 to 23 are the biased exponent. This bias allows positive and negative exponents to be stored as non-negative integers, for easier magnitude comparisons. The mantissa starts from bit 22 until the end (bit 0). When normalized, an implicit leading 1 is assumed ($1.M$), giving 24 bits of precision (Figure 2-4).

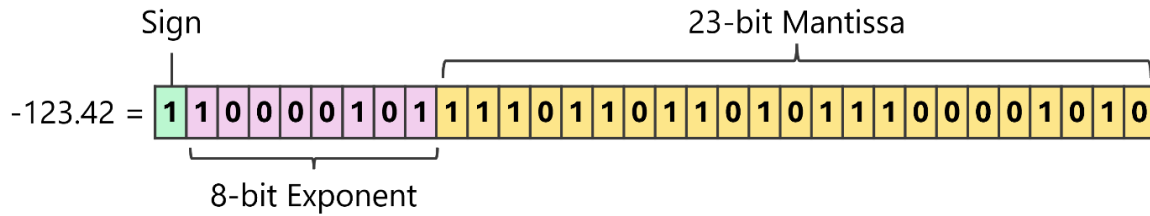


Figure 2-4: Representation of number -123.42 in IEEE 754 single-precision floating-point format.

Floating-point format can scale precision across extreme ranges and greatly reduces the risk of overflow. However, it consumes more resources than other alternatives, needs multi-stage pipelines and increases pipeline latency.

2.2.2. Arithmetic Architectures

Arithmetic architecture directly determines the quality and cost of a design, so it is an optimization problem that affects factors such as latency, timing, and numerical precision. The main architectures used in this thesis are analyzed below [14], [15], [16].

Multiplier-less Shift-and-Add Arithmetic

Multiplier-less designs can significantly reduce resource usage, so it is good practice to replace multipliers with shift-and-add architectures. Multiplication by a constant can be implemented by wire routing in the interconnect FPGA fabric, so it does not require logic gates, while addition still requires logic resources. The combination of them improves efficiency and reduces overall path complexity. Multiplication by powers of two (2^n) is implemented with left bit-shifts ($\ll n$) and division with right bit-shifts ($\gg n$) (Figure 2-5).

Arbitrary, non-power-of-two integers, are decomposed into sums and differences of shifted terms. For example, to multiply by 12, the factor is broken down to powers of two ($12 = 8 + 4$), so the result is the sum of two left shifts. For a number X : $12 \cdot X = (8 + 4) \cdot X = (2^3 + 2^2) \cdot X \equiv (X \ll 3) + (X \ll 2)$. This process does not require DSP blocks and can be implemented using only logic blocks.

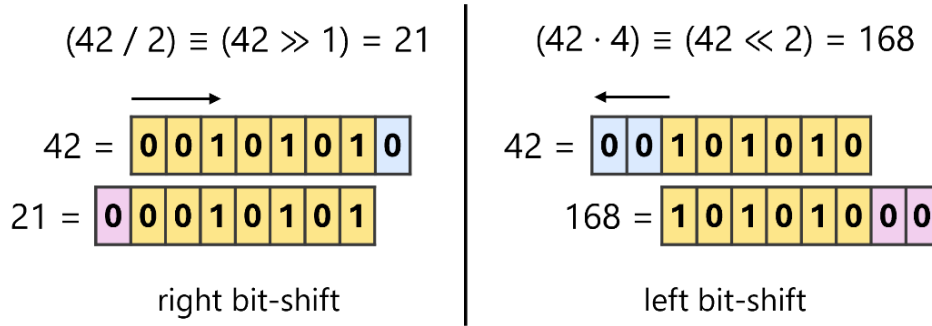


Figure 2-5: Example of bit-shifts used for multiplication and division with powers of two.

Adder Tree Structures

For calculations that require the sum of multiple operands, serial accumulation can create long critical paths that affect performance. To prevent this, additions can be organized into a multi-stage Adder Tree which shortens the critical path and reduces latency compared to serial accumulation. For M operands, the tree depth is approximately: $d = \lceil \log_2(M) \rceil$.

In each stage, operands are added in pairs in parallel in one cycle. The new sums are paired again in the following stage, with a valid signal propagating alongside them to define each cycle. In the case of an odd number of operands, where one cannot be paired, it passes through a delay register so the value can be maintained for one cycle and added later.

Multipliers

Shift-and-add only works when one operand is a known constant. When both operands are variable, the shifts are no longer fixed, so a full hardware multiplier is required. This is also the case for multiplication in floating-point arithmetic.

Multipliers are usually mapped onto DSP blocks, that are designed for high-speed numeric computations, so they can improve speed, and increase the achievable clock frequency. They can also be implemented using LUTs, especially for operands of smaller width or when there are no other DSP resources. This approach sums partial products using adder trees, so it naturally has a higher logic resource cost.

IEEE 754 Floating-Point Arithmetic

Calculations on single-precision IEEE 754 numbers cannot be carried out with shifts or a single addition, and they are typically implemented as multi-stage pipelines. Each operation works on the components of the operands. In addition and subtraction, the mantissas are aligned based on the difference of the exponents, while in multiplication the exponents are added and the mantissas are multiplied. In both cases the result is then normalized and rounded before it is reassembled into the standard format. Consequently, floating-point datapaths consume considerably more LUTs, flip-flops, and DSP slices than integer equivalents.

The implementations of floating-point addition and multiplication developed for this thesis are analyzed extensively in section 4.2.4.

2.3. Spatial Filtering

2.3.1. Spatial Convolution

Two-dimensional (2D) spatial filtering, used in image processing, is a technique that produces the output of a pixel value directly on the image grid, based on the pixel and its neighbours' initial values. A small square matrix of coefficients slides over every pixel of the image and performs calculations through convolution. The matrix is also named kernel, mask or window.

Convolution (symbol: $*$) is a mathematical operation on two functions w and f , that results in a third function g , a modified version of f . Spatial convolution for two dimensions is mathematically described by the equation [2-10].

$$g(x, y) = w(x, y) * f(x, y) = (w * f)(x, y) = \sum_{n_1=-a}^a \sum_{n_2=-b}^b w(n_1, n_2) \cdot f(x - n_1, y - n_2) \quad [2-10]$$

In image processing, the functions have two spatial variables (x, y) and they represent the image f of dimensions $(W \times H)$ and the filter w , with the filtered image g being the result. The filter is a kernel of $K \times K$ dimensions, with a radius of $a = b = \frac{K-1}{2}$. The kernel is centered with the target pixel, to compute the element-wise product of all elements that it overlaps with, and then it accumulates these products to determine the filtered value of the pixel, as shown in Figure 2-6.

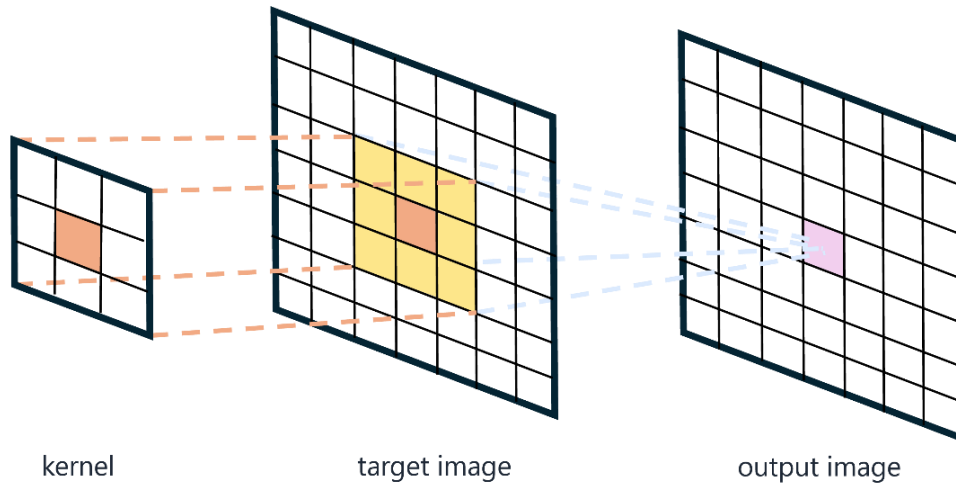


Figure 2-6: Schematic of image convolution with a 3x3 kernel.

When the window targets pixels at the edge of the image, it protrudes outside of the borders and there are no neighbor values on some of the sides. This problem is solved by restricting the calculations of the window to areas where it will be completely full, resulting in an output filtered image with reduced dimensions $(W - K + 1) \times (H - K + 1)$. Another approach is to add border padding to the image before filtering, either by mirroring the values of the sides or by adding a constant (zero) frame. This increases the input image dimensions by $P = \frac{K-1}{2}$ on each side, and the output maintains the original dimensions.

2.3.2. Selected Filters

Sobel Filter is an edge detection algorithm that uses two directional kernels, which are convolved with the image to produce partial gradients. The horizontal kernel computes the gradient for the x-direction G_x and the vertical for y-direction G_y . The partial gradients are squared, added and the exact gradient value is the square root as described in equation [2-11]. To ensure faster and simpler calculations, this is replaced by adding the absolute values of the partial gradients based on the equation [2-12]. This approximation is commonly used in Sobel edge detection applications [17].

$$\text{Exact Gradient Magnitude:} \quad G = \sqrt{G_x^2 + G_y^2} \quad [2-11]$$

$$\text{Approximate Gradient Magnitude:} \quad G = |G_x| + |G_y| \quad [2-12]$$

The directional kernels have different coefficients, depending on the size of the kernel ($K \times K$), where K is always an odd number. For two common cases, 3×3 and 5×5 windows, the values are presented below.

3×3 convolution kernels:	$K_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$	$K_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$
5×5 convolution kernels:	$K_x = \begin{bmatrix} -1 & -2 & 0 & 2 & 1 \\ -4 & -8 & 0 & 8 & 4 \\ -6 & -12 & 0 & 12 & 6 \\ -4 & -8 & 0 & 8 & 4 \\ -1 & -2 & 0 & 2 & 1 \end{bmatrix}$	$K_y = \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 2 & 8 & 12 & 8 & 2 \\ 0 & 0 & 0 & 0 & 0 \\ -2 & -8 & -12 & -8 & -2 \\ -1 & -4 & -6 & -4 & -1 \end{bmatrix}$

2-13: Coefficient values for the directional kernels of 3×3 and 5×5 Sobel filter.

To calculate the gradient for a pixel in the image the matrix of the neighbor pixels is convolved with the directional kernels. Detailed calculations for the two cases are presented by equations [2-14] and [2-15], where the target pixel is the center one: P_{11} for the 3×3 kernel, P_{22} for the 5×5 .

$x\text{-direction: } \begin{bmatrix} P_{00} & P_{01} & P_{02} \\ P_{10} & P_{11} & P_{12} \\ P_{20} & P_{21} & P_{22} \end{bmatrix} * K_x \Rightarrow G_x = (P_{00} + 2 \cdot P_{10} + P_{20}) - (P_{02} + 2 \cdot P_{12} + P_{22})$
$y\text{-direction: } \begin{bmatrix} P_{00} & P_{01} & P_{02} \\ P_{10} & P_{11} & P_{12} \\ P_{20} & P_{21} & P_{22} \end{bmatrix} * K_y \Rightarrow G_y = (P_{00} + 2 \cdot P_{01} + P_{02}) - (P_{20} + 2 \cdot P_{21} + P_{22})$

2-14: Convolution of image pixels with directional kernels to calculate the partial gradients in 3×3 Sobel filter.

x-direction:	$\begin{bmatrix} P_{00} & P_{01} & P_{02} & P_{03} & P_{04} \\ P_{10} & P_{11} & P_{12} & P_{13} & P_{14} \\ P_{20} & P_{21} & P_{22} & P_{23} & P_{24} \\ P_{30} & P_{31} & P_{32} & P_{33} & P_{34} \\ P_{40} & P_{41} & P_{42} & P_{43} & P_{44} \end{bmatrix} * K_x \Rightarrow$	$G_x = (2 \cdot P_{03} + P_{04} + 8 \cdot P_{13} + 4 \cdot P_{14} + 12 \cdot P_{23} + 6 \cdot P_{24} \\ + 8 \cdot P_{33} + 4 \cdot P_{34} + 2 \cdot P_{43} + P_{44}) \\ - (P_{00} + 2 \cdot P_{01} + 4 \cdot P_{10} + 8 \cdot P_{11} + 6 \cdot P_{20} + 12 \cdot P_{21} \\ + 4 \cdot P_{30} + 8 \cdot P_{31} + P_{40} + 2 \cdot P_{41})$
y-direction:	$\begin{bmatrix} P_{00} & P_{01} & P_{02} & P_{03} & P_{04} \\ P_{10} & P_{11} & P_{12} & P_{13} & P_{14} \\ P_{20} & P_{21} & P_{22} & P_{23} & P_{24} \\ P_{30} & P_{31} & P_{32} & P_{33} & P_{34} \\ P_{40} & P_{41} & P_{42} & P_{43} & P_{44} \end{bmatrix} * K_y \Rightarrow$	$G_y = (P_{00} + 4 \cdot P_{01} + 6 \cdot P_{02} + 4 \cdot P_{03} + P_{04} + 2 \cdot P_{10} \\ + 8 \cdot P_{11} + 12 \cdot P_{12} + 8 \cdot P_{13} + 2 \cdot P_{14}) \\ - (2 \cdot P_{30} + 8 \cdot P_{31} + 12 \cdot P_{32} + 8 \cdot P_{33} + 2 \cdot P_{34} \\ + P_{40} + 4 \cdot P_{41} + 6 \cdot P_{42} + 4 \cdot P_{43} + P_{44})$

2-15: Convolution of image pixels with directional kernels to calculate the partial gradients in 5×5 Sobel filter.

Gaussian Filter is a 2D convolution filter that blurs images, essentially smoothing details and reducing noise. It is defined by a 2D circularly symmetric Gaussian distribution, and the kernel is bell-shaped [18].

$$G(x, y) = \frac{1}{2\pi \cdot \sigma^2} \cdot e^{\frac{-(x^2+y^2)}{2\sigma^2}} \quad [2-16]$$

where σ is the standard deviation of the distribution and x, y the distances from the center pixel.

For image processing applications, the equation can be reduced to a finite window, by sampling in integer positions, $x, y \in \{-1, 0, 1\}$ for a 3×3 window and $x, y \in \{-2, \dots, 2\}$ for a 5×5 window. To maintain the sum equal to one, the equation changes as described in [2-17].

$$w(x, y) = \frac{e^{\frac{-(x^2+y^2)}{2\sigma^2}}}{\sum e^{\frac{-(x'^2+y'^2)}{2\sigma^2}}} \quad [2-17]$$

To implement the Gaussian filter computationally for the integer arithmetic, the continuous distribution is turned into integer coefficients with a power-of-two normalization factor, so the division reduces to a right shift. Each value of the kernel represents the weight the respective pixel will have in the calculation of the output. Consequently, the center pixel has the highest weight, and the weights decrease toward the edges, with the corner coefficients being the smallest. The coefficients are divided by their sum, so the normalized weights add up to one. The integer kernels presented in [2-18], are an approximation of the continuous Gaussian.

Like the Sobel filter, the filtered value for a pixel in the image, is the result of the convolution of the $K \times K$ matrix of the neighbor pixels and the respective kernel. Detailed calculations for the two cases are presented in equation [2-19], where the target pixel is the center one: P_{11} for the 3×3 kernel, P_{22} for the 5×5 .

3 × 3 convolution kernel:	$K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$
5 × 5 convolution kernel:	$K = \frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$

2-18: Coefficient values for the kernels of 3 × 3 and 5 × 5 Gaussian blur.

$\begin{bmatrix} P_{00} & P_{01} & P_{02} \\ P_{10} & P_{11} & P_{12} \\ P_{20} & P_{21} & P_{22} \end{bmatrix} * K \Rightarrow G = \frac{1}{16} \cdot (P_{00} + 2 \cdot P_{01} + P_{02} + 2 \cdot P_{10} + 4 \cdot P_{11} + 2 \cdot P_{12} + P_{20} + 2 \cdot P_{21} + P_{22})$
$\begin{bmatrix} P_{00} & P_{01} & P_{02} & P_{03} & P_{04} \\ P_{10} & P_{11} & P_{12} & P_{13} & P_{14} \\ P_{20} & P_{21} & P_{22} & P_{23} & P_{24} \\ P_{30} & P_{31} & P_{32} & P_{33} & P_{34} \\ P_{40} & P_{41} & P_{42} & P_{43} & P_{44} \end{bmatrix} * K \Rightarrow G = \frac{1}{256} \cdot (P_{00} + 4 \cdot P_{01} + 6 \cdot P_{02} + 4 \cdot P_{03} + P_{04} + 4 \cdot P_{10} + 16 \cdot P_{11} + 24 \cdot P_{12} + 16 \cdot P_{13} + 4 \cdot P_{14} + 6 \cdot P_{20} + 24 \cdot P_{21} + 36 \cdot P_{22} + 24 \cdot P_{23} + 6 \cdot P_{24} + 4 \cdot P_{30} + 16 \cdot P_{31} + 24 \cdot P_{32} + 16 \cdot P_{33} + 4 \cdot P_{34} + P_{40} + 4 \cdot P_{41} + 6 \cdot P_{42} + 4 \cdot P_{43} + P_{44})$

2-19: Convolution of image pixels with the kernels to calculate the gradients in 3 × 3 and 5 × 5 Gaussian blur.

For a precise application of the filter, the exact coefficients are computed using equation [2-17] with the standard deviations: $\sigma = \frac{1}{\sqrt{2}}$ for the 3 × 3 window and $\sigma = 1$ for the 5 × 5 window. Table 2-1 lists the resulting weights of the kernels, that are convolved with the image as described above.

Table 2-1: Exact coefficient values for the Gaussian filter kernels.

Window	Position	Value
3 × 3, $\sigma = \frac{1}{\sqrt{2}}$	(0,0)	0.331910665
	(0,1)	0.12210311
	(1,1)	0.044919224
5 × 5, $\sigma = 1$	(0,0)	0.162102822
	(0,1)	0.098320328
	(1,1)	0.059634294
	(0,2)	0.021938231
	(1,2)	0.01330621
	(2,2)	0.002969017

2.4. Tools & Development Methodology

In software architectures, the flow of instructions is described using programming languages (e.g. Python) and the processor executes them sequentially. Designing circuits for an FPGA is different and typically requires Hardware Description Languages. The code describes real physical hardware, where many logic gates and registers work simultaneously in parallel.

2.4.1. Hardware Description Methods

Hardware description languages (HDL) look like a programming language but describe a circuit instead of a sequence of instructions. HDL code statements are continuously active and are executed in parallel by default, not one after the other. The order in which they are written does not determine the order in which they run. Also, the language has time expressions to describe clock edges, delays in signals and the sequence of operations.

The main HDL languages are:

- **VHDL** (VHSIC Hardware Description Language) was standardized first, is stricter and requires verbose code.
- **Verilog** is simpler and has a more compact syntax. It allows combination of RTL design and testbench modules, while also introducing non-blocking assignments for a better description of synchronous circuits.
- **SystemVerilog** initially began as a set of extensions to Verilog, but later they were merged, so Verilog is now a part of SystemVerilog. It introduced combinational (always_comb) and sequential (always_ff) logic blocks and the “logic” data type for cleaner signal connections.

High-level synthesis (HLS) is a design methodology that translates code written in high-level languages (e.g. in C or C++) into RTL designs ready for hardware implementation. It decides how many adders, multipliers, and memories to instantiate and assigns each operation to a clock cycle. While this speeds up development, the designer loses direct control over clock cycles, and unnecessary FPGA resources are often used [20].

2.4.2. Xilinx Vivado Workflow

The AMD Xilinx Vivado Design Suite is a tool used to design, simulate, synthesize and implement hardware designs. The complete design flow includes the following stages:

1. **Design.** The SystemVerilog code files of the filter, the supporting modules and the testbench are added to the project. The platform sets the module hierarchy and shows any syntax errors.
2. **Behavioral simulation.** The design and the testbench are executed by the Vivado simulator. The input file is used to stream pixel values cycle by cycle and the output text file with the filtered pixels is created. Also, latency and total cycle count are measured. Simulation verifies the correctness of the design, the result is a set of waveforms for all signals, essentially showing the logical functionality of the RTL design in response to the testbench stimuli. It does not consider actual timing or delays (Figure 2-7).
3. **Timing constraints.** A constraints file (format: XDC - Xilinx Design Constraints) is added to the project to define the clock constraints. In this file, the target clock period (T_{target}) is set, that determines the upper speed limit of the architecture. The platform tries to optimize the design for this period, and if the target is met, optimization stops, but if it is very strict, all efforts for optimization will still result in

a negative slack. The Worst Negative Slack (WNS) and the Maximum Operating Frequency ($F_{\max}$) are calculated based on this constraint, so the same target period was used for every design in this thesis, to make their comparison fair.

4. **Synthesis.** The design is converted from a high-level to a gate-level description. The structures described in the code, such as registers, adders and multipliers, are assigned to physical structures (e.g. LUTs, flip-flops, DSP slices) on the device. This allows for an initial resource estimation, but it does not include the actual physical wire placement.
5. **Implementation.** The synthesized design is mapped on the target device. Logic gates are assigned to a specific physical position on the chip, and their connections are routed using the FPGA interconnect. A schematic view of the device is also generated, showing the slices' position on the target FPGA, as shown in Figure 2-8.
6. **Reports.** After implementation, various reports provide the resource cost and performance metrics of the design. The utilization report lists the number of LUTs, flip-flops and DSP slices used. The timing summary gives the worst negative slack, from which other performance metrics are calculated.

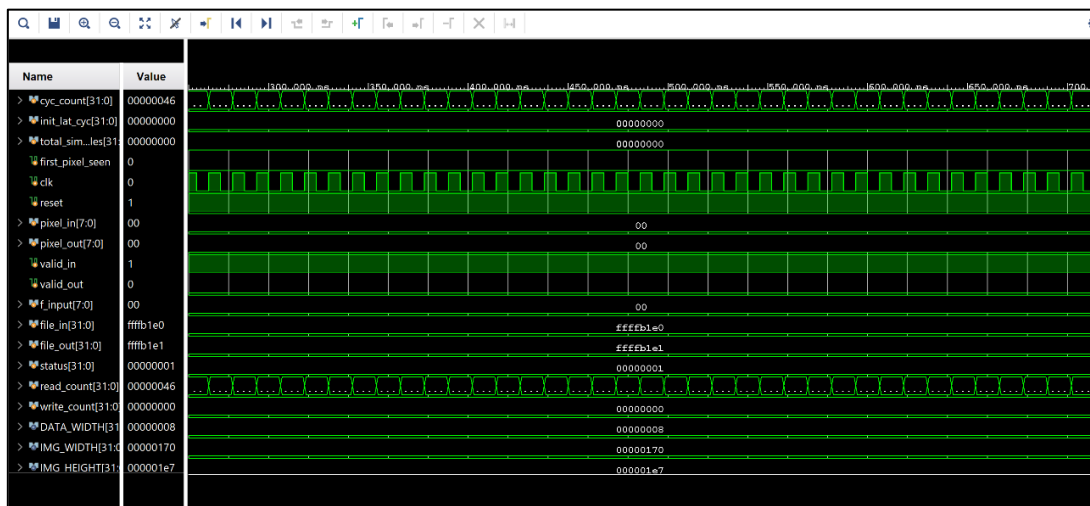


Figure 2-7: Screenshot of the signal waveforms from Vivado Behavioral simulation.

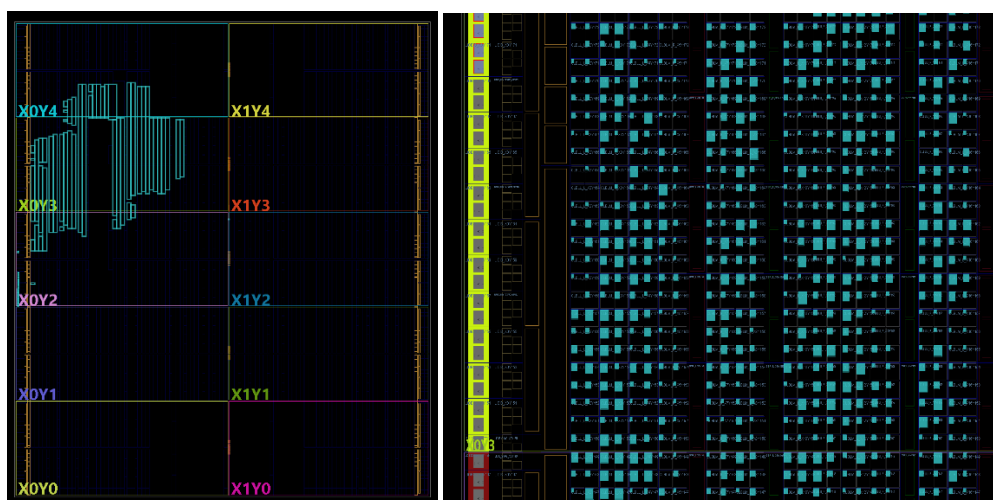


Figure 2-8: Post-implementation device view and zoomed in details, showing placed and routed slices on clock regions of the target FPGA.

2.5. Comparison & Evaluation Metrics

2.5.1. Image Quality and Similarity

In order to assess the correctness of an RTL image processing implementation, the reconstructed hardware output is compared to a software-generated reference image. The following image quality metrics are used to evaluate the similarity and detect possible errors.

Maximum Absolute Difference (MAD) shows the localized difference between the two images. Even though it is not a good measure for structural accuracy, it pinpoints exactly rounding errors or misalignments between the result and the reference.

$$MAD = \max\{ |I_1(x, y) - I_2(x, y)| \} \quad [2-20]$$

where I_1, I_2 are the two images with dimensions $(W \times H)$.

The **Mean Squared Error (MSE)** and **Root Mean Squared Error (RMSE)** are indicators for the level of squared error between two images. They are calculated by the squared difference of each pixel of the reference and the result. A lesser value suggests better similarity [19]. These approaches disproportionately penalize high isolated errors compared to uniform noise, because of their quadratic form.

$$MSE = \frac{1}{W \cdot H} \cdot \sum_{x=0}^W \sum_{y=0}^H (I_1(x, y) - I_2(x, y))^2 \quad [2-21]$$

$$RMSE = \sqrt{MSE}$$

where I_1, I_2 are the two images with dimensions $(W \times H)$.

Peak Signal-to-Noise Ratio (PSNR) quantifies the reconstruction quality of the final image relative to the maximum value of the reference and is measured in [dB] [19]. High PSNR values correspond to lower reconstruction errors, with complete resemblance being measured as $PSNR = \infty$. Since it is based on the mean squared error, it cannot detect errors in structure.

$$PSNR = 10 \cdot \log_{10} \frac{MAX^2}{MSE} \quad [2-22]$$

where $MAX = 255$, the maximum pixel value for an 8-bit grayscale image.

Structural Similarity Index Measure (SSIM) is a more objective image quality metric, because it compares the combination of three factors: loss of correlation, luminance distortion and contrast distortion, as shown in equation [2-23]. The luminance term compares the local means, the contrast term compares the local variances, and the structure term compares the covariance. The constants C_1 and C_2 keep the expression stable when the denominators are close to zero. To calculate SSIM for the entirety of the image, a $K \times K$ sliding window scans the image calculating a value $SSIM_j$ at each position, and the overall index is the average of these values [21].

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1) \cdot (2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1) \cdot (\sigma_x^2 + \sigma_y^2 + C_2)} \quad [2-23]$$

$$MSSIM(X, Y) = \frac{1}{M} \cdot \sum_{j=1}^M SSIM(x_j, y_j)$$

where μ_x, μ_y are mean intensities, σ_x^2, σ_y^2 the variances, σ_{xy} their covariance, C_1, C_2 are stabilizing constants, X and Y are the images, x_j and y_j are the image contents at the j^{th} window and M is the number of windows in the image.

2.5.2. Hardware & Performance Metrics

To evaluate the efficiency of an FPGA-based spatial image processing architecture the physical resources used need to be measured. Another key metric is timing analysis, that verifies that the signal is transferred through the combinational logic paths without violating the time constraints of the setup. Other important factors are operational latency, frame rate and streaming throughput [22]. The factors used for architecture evaluation, also analysed in [2.1.2], [2.1.3], are listed below.

Look-Up Tables (LUTs) are the primary building blocks of combinational logic in FPGAs. They are responsible for implementing Boolean logic functions and arithmetic like adders and subtractors, so they are a measure of combinational logic cost in a circuit.

Flip-Flops (FFs) are binary storage elements that also register data from input to output at every clock edge and define pipeline stages. The total flip-flop count quantifies the sequential operation cost and pipeline depth of the design.

Digital Signal Processing Blocks (DSPs) are arithmetic blocks, optimized for high-speed multiply-accumulate (MAC) operations. DSP block consumption is an important distinction between multiplier-less integer datapaths and custom floating-point pipelines.

Worst Negative Slack (WNS) is the smallest timing margin across all paths, respective to the target clock period. A positive slack ($WNS \geq 0$) shows that timing constraints are met, while timing violations have a negative value ($WNS < 0$).

Maximum Operating Frequency ($F_{\max}$) is the maximum theoretical clock rate that can be achieved by the circuit without causing timing failure.

$$F_{\max} = \frac{1}{T_{\text{target}} - WNS} \quad [2-24]$$

where T_{target} is the target clock period and WNS is the Worst Negative Slack.

Latency (L) is defined as the number of clock cycles that pass from the first valid input pixel until the datapath produces the first valid output pixel.

Frame Rate (FPS) is measured as frames produced per second, the rate of data processing.

$$FPS = \frac{F_{\max}}{\text{Total Cycles per Frame}} \quad [2-25]$$

where $F_{\max}$ is the maximum operating frequency.

Throughput (T) is calculated from the maximum operating frequency and the number of pixels produced per cycle, which by design is one. It is a metric of the data processing rate of the design.

$$T = \text{pixels per cycle} \cdot F_{\max} \quad [2-26]$$

where $F_{\max}$ is the maximum operating frequency.

CHAPTER 3: Literature Review

Modern computer vision applications have increasing computational demands. This has led to extensive research on hardware accelerators that can deliver high throughput under strict constraints.

Image processing operations, and more specifically spatial filtering, require multiple computations that increase with image resolution and frame rate. In real-time spatial filtering, pixels must be processed at the same rate that the frames arrive. A high-definition video creates a stream of over a hundred million pixels per second. A convolution with a 3x3 matrix performs nine multiply-accumulate operations for each output pixel, which equals more than 1 billion multiply-accumulate operations per second. This makes software-only approaches generally inefficient [23].

CPUs, GPUs, and FPGAs handle this workload with different architectural mechanisms. Multi-core CPUs use SIMD (Single Instruction, Multiple Data) instructions, while GPUs have many parallel and simpler computing cores. FPGAs allow customized datapaths that stream pixels directly between processing stages, without the need for extra storage, keeping data on chip. A comparison of vision kernels executed on CPU, GPU and FPGA, each running its vendor optimised library, showed that while the GPU is more energy efficient for simple kernels, the FPGA worked better for complex kernels and complete pipelines, with a reduction of energy per frame by 1.2 to 22.3 times. This advantage increases as pipeline complexity grows [2].

Timing behaviour differs as well. On a processor, the execution time of a frame can change due to different techniques used, while in hardware, the computations are based on the clock cycle, so the latency is fixed and can be calculated at the design [2]. Also, instead of executing instructions serially, FPGA architectures use bit-level, pixel-level, neighbourhood-level, and task-level parallelism [23].

A recurring observation from the literature is that FPGA implementations are particularly effective for convolution-based operations, where the same arithmetic structure is repeatedly applied across an image. As a result, spatial filtering, image enhancement, denoising, feature extraction, and edge detection are from the most frequently investigated FPGA image processing applications.

These studies show that FPGAs are a suitable choice for image processing, and the advantage depends on the architecture and the size of the workload. While their experimental approaches differ, they do not vary the number format, the arithmetic approximation and the kernel size while keeping the device, the filter architecture and the reference image fixed. Without that, the effect of an arithmetic choice on resources, operating frequency, latency, throughput and output accuracy cannot be separated from the effect of the platform.

This chapter reviews the published work behind these decisions and identifies the gaps that motivate the present thesis.

3.1. Hardware architecture for spatial convolution

Two-dimensional convolution is the core of several image processing algorithms. Each output pixel calculation depends on neighbouring input pixels, so efficient data access and storage mechanisms are critical. Early FPGA implementations used full-frame buffering, that is, storing entire images before processing. These methods create excessive memory requirements, especially for high-resolution and real-time applications. As a result, streaming architectures based on line-buffers are currently the main solution, because they provide access to the neighbouring pixels without the need for full-frame storage [12].

Line buffers store only the image information required to fill the current convolution window, which amounts to $K-1$ image rows for a $K \times K$ window. The pixels are processed continuously as they arrive through a shift register that produces the window [12], [23], [24]. Typical implementations construct sliding windows of 3×3 , 5×5 , or larger dimensions through a combination of line buffers and shift registers [25], [26], [27].

The general problem of transforming a serial software algorithm so that it fits a hardware model has been addressed. Low-level image processing algorithms must be transformed to pipelined architectures using line buffers and shift registers that process pixels synchronously [28]. These architectures reduce repeated memory accesses by reserving neighborhood pixels that will be reused on-chip, and once the initial window is filled, the design can accept a new pixel on each clock cycle [22], [28].

However, these implementations examine individual architectural configurations without providing a controlled comparison across kernel size, numerical format, and pipeline depth. Consequently, their results do not fully quantify how these design parameters affect resource consumption, operating frequency, latency, throughput, and numerical accuracy.

3.2. FPGA implementations for spatial filtering

The Sobel operator appears repeatedly in the FPGA literature, because it is less sensitive to noise compared to simpler gradient operators, and it is easier to implement. For the 3×3 kernel, the coefficients are 0, ± 1 and ± 2 (1 and 2 are powers of 2), which means that multiplication can be implemented using sign changes, direct connections, and one-bit shifts, reducing the operation count. A 2014 review identified more than twenty hardware implementations of the Sobel operator and compared architectural choices [29]. Four architectures for the gradient computation were implemented: a sequential design using one processing element, a pipelined version of it, a parallel design with two processing elements, and a combination of pipelining and parallelism. Across the four designs, LUT usage and frame rate were measured, which quantifies the trade-off between area and performance. However, all designs keep the arithmetic precision constant. They use the same integer data path and the same shift-based calculation of the coefficients, and the outputs are compared to each other to verify they are identical, not to a reference image. As a result, output quality is constant by construction and cannot act as a design variable. This comparison quantifies the cost of the architectural choices but does not report on the cost of the arithmetic.

Other work follows the same approach, comparing various edge detection operators on identical hardware, and calculating the resource usage, for example, Sobel [30], Prewitt and Roberts [31], and Canny [32]. In other approaches, the operator is fixed while the surrounding architecture is optimised [29], or the operator is reformulated to be more hardware-friendly [33]. Another variation changes the arithmetic instead of the architecture, reducing the datapath to eight bits [34].

More recently, Ravichandran et al. [17] implemented a modified Sobel operator in Verilog, using eight directional kernels instead of two and an 8-bit datapath with bit-shifts instead of multiplications. The output was compared to Prewitt, Roberts and standard Sobel filters, using mean squared error, normalised cross-correlation, average difference and normalised absolute error. The error metrics show the difference between the edge map and the original image, not between the hardware output and an implementation of the same filter.

The Gaussian filter is more demanding because its kernel has non-integer values that need to be approximated, which introduces errors. In one approach, the Gaussian kernel is quantized to unsigned fixed-point format with a fractional part or represented directly with IEEE 754 floating-point arithmetic [18]. Another strategy is to approximate the filter coefficients by powers of two, which allows multiplications to be replaced

with shift operations and reduces hardware complexity, while maintaining filter quality comparable to conventional Gaussian filters [35]. The same strategy is applied to non-linear filtering, by approximating the weight values and matching them to a piecewise lookup table [36]. Compared with other FPGA implementations of the same filter, the resulting design reduced resource requirements by up to 70 percent.

3.3. Fixed-point and floating-point arithmetic

The choice of numerical representation is one of the most important design decisions in FPGA systems. Floating-point arithmetic provides large precision across extreme ranges, however, it requires more hardware resources than other alternatives and often increases latency [18]. Floating-point representations follow the IEEE 754 standard which fixes the sign, exponent and significand field widths, so the designer selects a precision rather than sizing the format [37].

Fixed-point arithmetic has traditionally been preferred for FPGA image processing because it offers lower resource utilization, faster execution, and simpler hardware structures. Resource savings can be significant, especially in spatial convolution applications where heavy multiplication and accumulation operations are involved. However, there is no universal equivalent hardware representation for the fixed-point formats, so the the number of integer and fractional bits must be chosen for each variable. These choices determine both the hardware cost and the quantisation error. The wordlengths have been analytically modeled to determine optimal width that meets standard error thresholds, while reducing logic gate usage [38]. In other cases, wordlength allocation was automatically determined based on constraints for output noise [39], and analytical methods were developed to evaluate output accuracy directly from system functions, without requiring repetitive simulations [40].

The language in which a filter is described also affects the reported results. An evaluation of FPGA high-level synthesis tools found differences in the quality of results between the tools because of different internal optimizations [41]. A review of 46 comparative studies found that RTL still produces better results on average, with HLS usually consuming more basic FPGA resources, but reducing design time significantly [42]. Specifically for image processing applications, HLS and RTL have been shown to reach comparable performance [43].

3.4. Kernel size scaling and cascaded filter errors

Most of the implementations reviewed above report a single window size [17], [25], [26], [34], so the cost of enlarging the window cannot be recovered from the figures given. When different kernel sizes are reported, the results are usually measured as a ratio of the overall resource consumption without quantifying the exact physical resources. For example, a 5×5 filter has been shown to occupy 1.76 times fewer resources than a 7×7 version and 4.16 times fewer than an 11×11 version [36].

Cascaded filtering pipelines introduce other numerical problems. For example, a Canny edge detector begins with Gaussian smoothing and passes the result to a Sobel edge detection computation. In FPGA implementations of the complete chain, performance and accuracy of the output are often evaluated at the final stage, using visual comparison or quality measures [44]. The propagation of quantisation error from the first to the second stage is not tracked.

Analytical methods to evaluate the propagation of quantisation error through sequences of operations have been proposed [39], [40], [45], but they assume linear time-invariant systems and have been mainly applied in 1D digital signal processing structures. Consequently, they do not measure multi-dimensional error accumulation that exists in cascaded 2D vision pipelines.

3.5. Motivation for this thesis

The reviewed literature confirms that FPGAs are a suitable choice for spatial image processing, but most implementations vary either the filter operator or the hardware architecture, while keeping the numerical representation fixed. Other studies typically change the target device or hardware description method (HLS, RTL). Consequently, the effects of numeric representation (fixed-point, floating-point), arithmetic architecture (shift-and-add, multipliers), and kernel size cannot be separated from other design characteristics.

One study does compare the two formats in a single design, reporting LUT usage for a 3×3 Gaussian filter for fixed-point and floating-point implementations on the same device [18]. The floating-point datapath is based on a VHDL library, and no DSP usage is reported. This shows one implementation case rather than the cost of the format itself. The larger floating-point configurations exceed the LUT capacity of the target device and are reported as synthesis estimates. Output accuracy is measured, but it is not correlated with the resource metrics, so the cost of a given accuracy cannot be assessed.

Also, studies exploring approximate arithmetic report quality metrics against baselines that differ. Some compare results relative to the unfiltered input, an existing conventional implementation, or against a software model but by measuring quality scores and not the actual output images. Consequently, the hardware cost of the arithmetic approximation cannot be defined.

This thesis presents a controlled comparison of 2D spatial filtering pipelines where device, HDL, architecture, test image and evaluation script are held fixed and one design parameter is varied at a time. This way, the trade-offs can be quantified between zero-DSP multiplier-less integer and custom binary32-format single-precision floating-point pipelines for both 3×3 and 5×5 Sobel and Gaussian kernels. Every configuration is compared against a Python equivalent of the same filter, which separates the error introduced by the hardware from the error inherent to the filter. The same setup is applied to a two-stage cascade of filters, so that the error from the first stage can be measured at the output of the second.

CHAPTER 4: Methodology

The methodology of this thesis is structured into three consecutive stages: input image preprocessing, hardware-accelerated filter execution for various architecture cases, and output image reconstruction and comparison, as described in Figure 4-1.

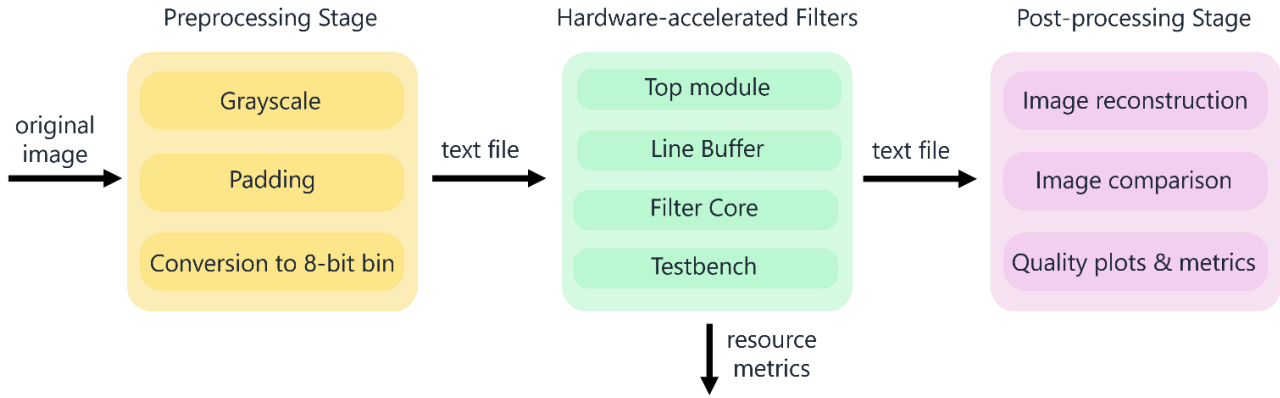


Figure 4-1: Overall workflow, including inputs and outputs.

4.1. Preprocessing

4.1.1. Preparation of input file

The preprocessing pipeline turns the original image into a suitable form for streaming into the hardware simulation testbench. The source image is initially transformed from RGB colors to 8-bit grayscale using the OpenCV library (cv2.cvtColor with the COLOR_BGR2GRAY color code).

To prevent boundary pixel information loss during the convolution, constant border padding is applied using the NumPy library (numpy.pad), as shown in Figure 4-2.

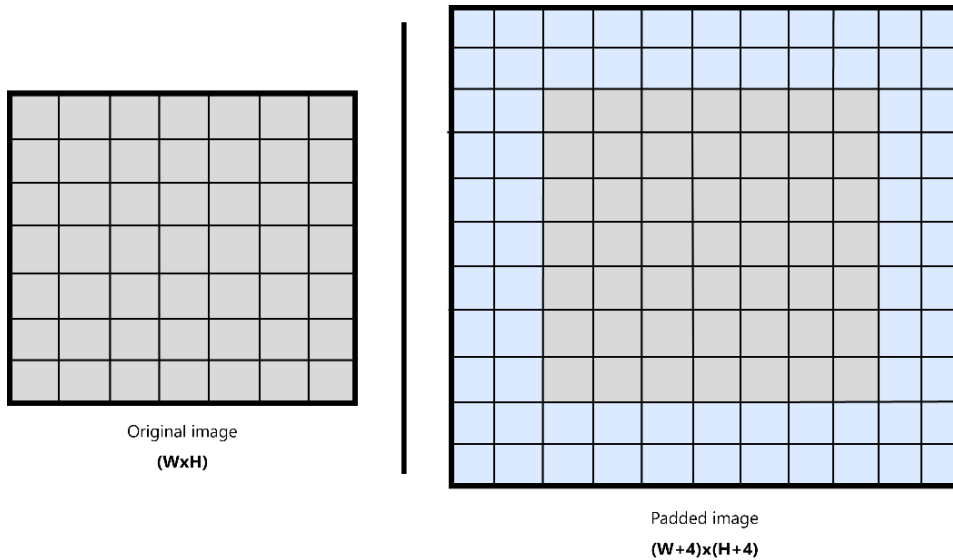


Figure 4-2: Schematic of an original $(W \times H)$ and a padded image $(W + 4) \times (H + 4)$, for cascaded filters with $K=3 \Rightarrow N=1$ or a stand-alone with $K=5 \Rightarrow N=2$.

A N -pixel margin with a constant value of zero (black) is added to all sides, expanding the dimensions of the original image from $W \times H$ to $(W + 2 \cdot N) \times (H + 2 \cdot N)$, and essentially creating a black frame. The padding width N is defined by the kernel size: $N = \frac{\text{Window Size}-1}{2}$, so a 1-pixel margin is added for 3×3 kernels and 2-pixel margin for 5×5 . This guarantees that during the filter application, when the kernel is processing values at the edge, no actual pixel information will be lost, and the reconstructed output will preserve the original $W \times H$ resolution. In the case of cascaded filters, the padding is $2 \cdot N$ pixels on each side $(W + 4 \cdot N) \times (H + 4 \cdot N)$, to account for the size reduction between the two filter stages.

Finally, the 2D padded image matrix is scanned (left-to-right, row-by-row). Each value is converted to its 8-bit binary representation and written line-by-line to the output text file, creating the input stream for the SystemVerilog testbench. For example, as shown in Figure 4-3, at the initial image at the 100th row and 100th column the values of the pixels are [0, 176, 80], while in the 1-pixel padded grayscale edition in the same relative position (meaning at the 101st row, 101st column due to 1 pixel padding) the value is 112, which then is converted to binary (01110000).

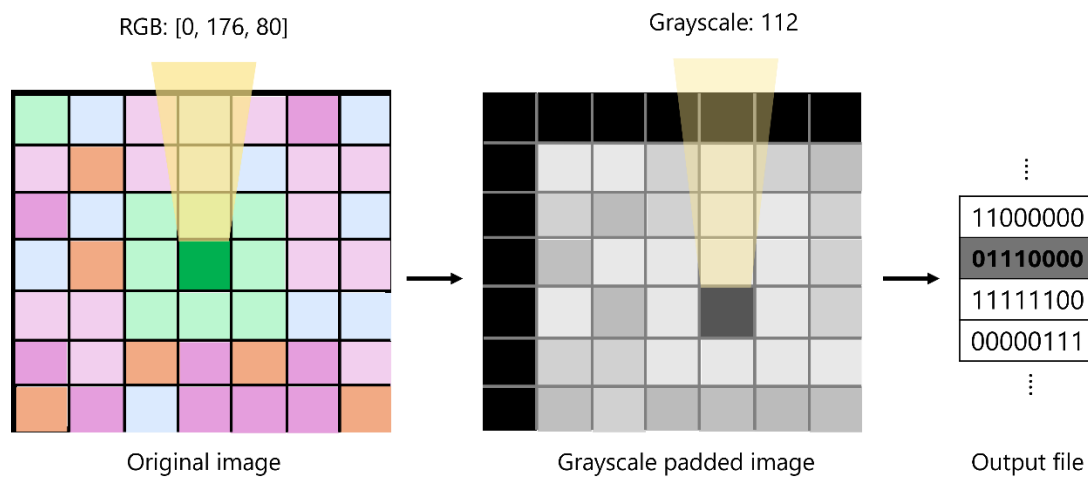


Figure 4-3: Process of converting original image to binary text file.

4.2. Filter execution in hardware

This section analyzes the RTL architecture of the modules used for the hardware-accelerated filters. One input pixel is processed, and one filtered output pixel is delivered on every clock cycle. The following modules are combined for the Sobel filter, the Gaussian blur and for a cascade of filters (Gaussian blur and then Sobel). The same process is followed for a 3×3 and 5×5 convolution window.

In general, every module is parted from the definition of the input and output wires and signals that interact with other modules, the parameters such as image width and height, and then the core body with the combinational and sequential logic.

4.2.1. Line Buffer Module

The line buffer module converts the serialized 1D pixel stream into the desired window matrix. A 3×3 convolution kernel requires intermediate storage of two complete image lines. Two line memories (line0 and line1), are used as circular buffers, with a shared position index (curr_pix_pos) that cycles through the

addresses $[0, W - 1]$. When a new pixel arrives as input, the validation signal is activated, and the oldest pixel is transferred from line1 into line0 at curr_pix_pos , while the new pixel (pixel_in) overwrites $\text{line1}[\text{curr_pix_pos}]$. The sliding window matrix is created using 8-bit registers connected with parallel shift chains, that update simultaneously when the valid_in signal is activated. The transformations for 3×3 window follow the logic presented in Table 4-1.

Table 4-1: Pixel value transformations for a 3x3 line buffer.

Row 0	Row 1	Row 2
$P_{00} \leq P_{01}$	$P_{10} \leq P_{11}$	$P_{20} \leq P_{21}$
$P_{01} \leq P_{02}$	$P_{11} \leq P_{12}$	$P_{21} \leq P_{22}$
$P_{02} \leq \text{Line}_0[\text{index}]$	$P_{12} \leq \text{Line}_1[\text{index}]$	$P_{22} \leq \text{input-pixel}$

This process flattens the 2D image coordinates, risking errors when transitioning to a new row (e.g. blending pixels of the right edge of line y with the left edge of line $y+1$). To prevent this, a coordinate tracker is implemented with two registers, for the current column and row (in_col , in_row). They track the position of the last input pixel, so the coordinates of the center pixel of the window they describe are $(\text{in_row} - 1, \text{in_col} - 1)$. The column tracker increases by one until one column before the end ($\text{in_col} \in [0, W - 1]$) and then resets to zero to signalize the change of row (Figure 4-4).

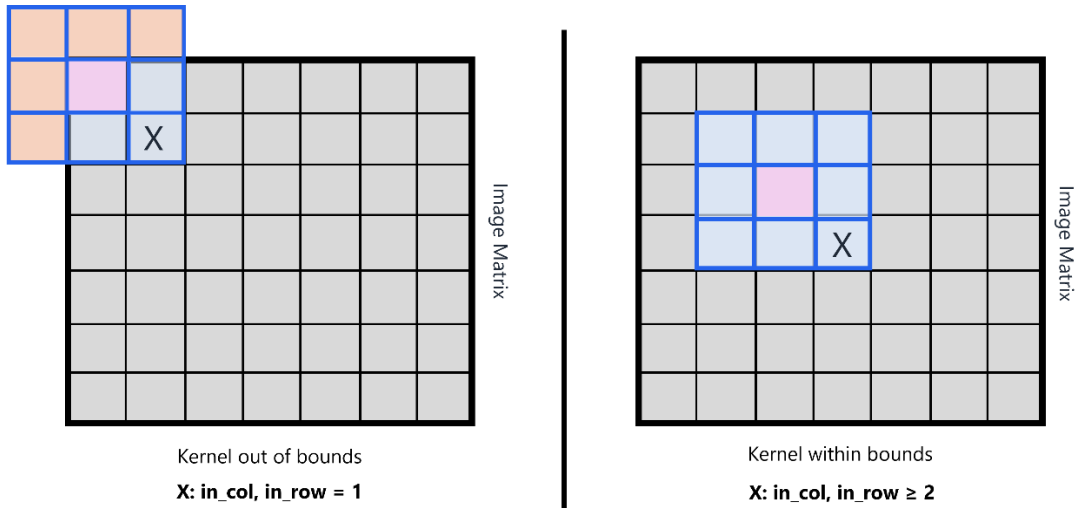


Figure 4-4: Representation of convolution kernel inside and out of the image limits and coordinate tracked pixel.

The output validation signal is activated after the warm-up latency (the window is filled), and only if the window is not at the edges. This translates to a pixel count larger than two full lines and two pixels ($\text{pixel_count} \geq W \cdot 2 + 2$), and to values of the coordinate trackers that form a window that does not contain padding values ($W - 1 \geq \text{in_col} \geq 2$, $H - 1 \geq \text{in_row} \geq 2$). Consequently, an input stream with padding $(W + 2) \times (H + 2)$ produces an output of exactly $(W \times H)$ pixels, completely filtering out the padding.

The line buffer for the 5×5 convolution kernel works identically, but on a larger scale. It utilizes four line memories ($K - 1$), to fill the window with 25 values. The warm-up latency is $W \cdot 4 + 5$ cycles, and the coordinate trackers are restricted to four positions from the edge ($W - 1 \geq \text{in_col} \geq 4$, $H - 1 \geq \text{in_row} \geq 4$) to account for the larger window size. This reduces the output stream by four pixels in each dimension, but since the input stream comes double padded $(W + 4) \times (H + 4)$, the output is exactly $(W \times H)$ pixels.

4.2.2. Top Module

The Top module functions as a wrapper that creates the internal dataflow between the Line Buffer and the Core Filter module, as shown in Figure 4-5.

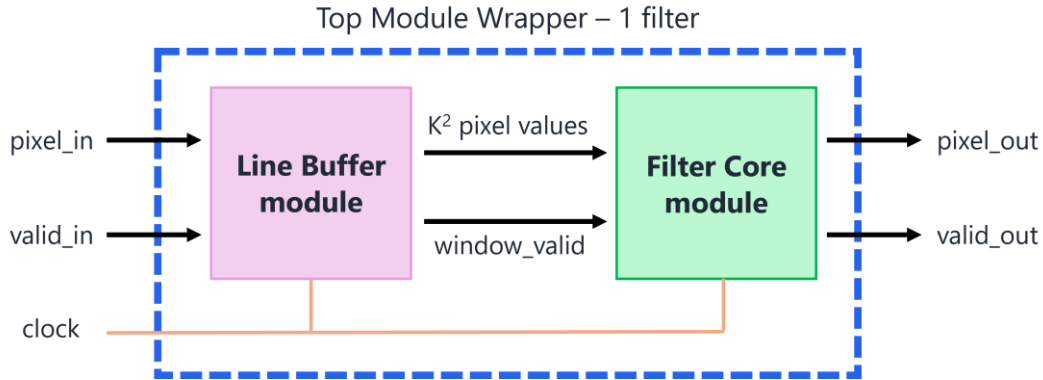


Figure 4-5: Architecture of the Top module for a stand-alone filter.

The image dimensions (IMG_WIDTH , IMG_HEIGHT) and the bit width ($DATA_WIDTH$) are passed as parameters, while it receives one input pixel ($pixel_in$) per cycle, along with the validation signal ($valid_in$).

To coordinate the data movement between the two modules, Top module streams the input pixel and the validation signal to the Line Buffer, which then generates the window pixel values and the signal that the window is valid ($window_valid$), only after it has passed the warm-up stage ($row, col \geq K - 1$). This serves as the initiation signal for the filter module, which receives the window values, performs calculations and returns the final pixel value ($pixel_out$) and the last validation signal ($valid_out$).

For the case of the cascaded filters (Figure 4-6), the Top module functions similarly, with the addition of another line buffer instance between the two filter cores, and a stage-by-stage “handshake” chain for the validation signals. Because the process inherently discards border margins, the initial image is double padded $(W + 4 \cdot N) \times (H + 4 \cdot N)$, and the second line buffer is initiated with decreased dimensions $(W + 2 \cdot N) \times (H + 2 \cdot N)$. This way the result will maintain the original image dimensions $(W \times H)$.

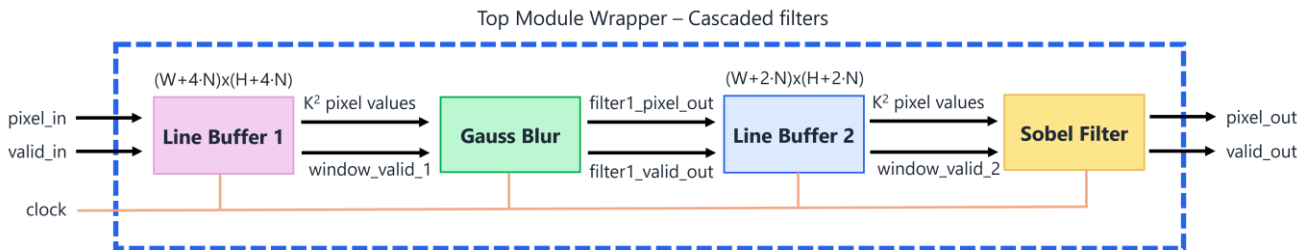


Figure 4-6: Architecture of the Top module for cascaded filters.

4.2.3. Integer Implementation

The implementation of the integer datapath uses only multiplier-less shift-and-add arithmetic. The filter weights are integers and powers of two, or sums of powers of two (e.g. Sobel: $\pm 1, \pm 2$, Gaussian blur: 1,2,4), so multiplication operations are performed with bitwise shifts ($A \ll 1 \equiv A \cdot 2$). These operations are

implemented with simple wiring on the FPGA, so they are executed in one cycle, and they don't require DSP blocks.

Input Expansion

The module receives the window pixel values as 8-bit unsigned integers ($P \in [0,255]$). All values are stored as signed integers: the input value padded with extra zeros in the front to avoid errors in overflow cases. The number of zeros in the padding depends on the kernel size and filter case, because higher weights result in bigger numbers.

Gradient Calculation

The padded values need to be multiplied with the corresponding filter weight, which happens by shifting the value left for specific bits. For example, multiplication by two is left-shift of 1 bit ($A \ll 1$), and multiplication by six is left-shift of 2 bits added with left-shift of 1 bit ($A \ll 2 + A \ll 1$). The results go through addition and other simple operations to calculate the gradients of each filter, in a single combinational block. The exact calculations for each filter are presented in Table 4-2. In the fixed-point implementations of the Gaussian blur, the approximated integer coefficients with a power-of-two normalization factor are used.

Table 4-2: Final gradient calculation using bit-shifts for Sobel and Gaussian filter, kernel sizes 3x3 and 5x5.

Filter	Shifts & Addition	Final Calculation
3x3 Sobel Filter	$G_x = (P_{02} + (P_{12} \ll 1) + P_{22}) - (P_{00} + (P_{10} \ll 1) + P_{20})$ $G_y = (P_{00} + (P_{01} \ll 1) + P_{02}) - (P_{20} + (P_{21} \ll 1) + P_{22})$	$G = G_x + G_y $
3x3 Gaussian Blur	$G_{sum} = P_{00} + (P_{01} \ll 1) + P_{02} +$ $+ (P_{10} \ll 1) + (P_{11} \ll 2) + (P_{12} \ll 1) +$ $+ P_{20} + (P_{21} \ll 1) + P_{22}$	$G = G_{sum} \gg 4$
5x5 Sobel Filter	$G_x = [(P_{03} \ll 1) + P_{04} + (P_{13} \ll 3) + (P_{14} \ll 2)$ $+ (P_{23} \ll 3) + (P_{23} \ll 2) + (P_{24} \ll 2) + (P_{24} \ll 1)$ $+ (P_{33} \ll 3) + (P_{34} \ll 2) + (P_{43} \ll 1) + P_{44}]$ $- [P_{00} + (P_{01} \ll 1) + (P_{10} \ll 2) + (P_{11} \ll 3)$ $+ (P_{20} \ll 2) + (P_{20} \ll 1) + (P_{21} \ll 3) + (P_{21} \ll 2)$ $+ (P_{30} \ll 2) + (P_{31} \ll 3) + P_{40} + (P_{41} \ll 1)]$ $G_y = [P_{00} + (P_{01} \ll 2) + (P_{02} \ll 2) + (P_{02} \ll 1)$ $+ (P_{03} \ll 2) + P_{04} + (P_{10} \ll 1) + (P_{11} \ll 3)$ $+ (P_{12} \ll 3) + (P_{12} \ll 2) + (P_{13} \ll 3) + (P_{14} \ll 1)]$ $- [(P_{30} \ll 1) + (P_{31} \ll 3) + (P_{32} \ll 3) + (P_{32} \ll 2)$ $+ (P_{33} \ll 3) + (P_{34} \ll 1) + P_{40} + (P_{41} \ll 2)$ $+ (P_{42} \ll 2) + (P_{42} \ll 1) + (P_{43} \ll 2) + P_{44}]$	$G = G_x + G_y $
5x5 Gaussian Blur	$G_{sum} = P_{00} + (P_{01} \ll 2) + (P_{02} \ll 2) + (P_{02} \ll 1)$ $+ (P_{03} \ll 2) + P_{04} + (P_{10} \ll 2) + (P_{11} \ll 4)$ $+ (P_{12} \ll 4) + (P_{12} \ll 3) + (P_{13} \ll 4) + (P_{14} \ll 2)$ $+ (P_{20} \ll 2) + (P_{20} \ll 1) + (P_{21} \ll 4) + (P_{21} \ll 3)$ $+ (P_{22} \ll 5) + (P_{22} \ll 2) + (P_{23} \ll 4) + (P_{23} \ll 3)$ $+ (P_{24} \ll 2) + (P_{24} \ll 1) + (P_{30} \ll 2) + (P_{31} \ll 4)$ $+ (P_{32} \ll 4) + (P_{32} \ll 3) + (P_{33} \ll 4) + (P_{34} \ll 2) + P_{40}$ $+ (P_{41} \ll 2) + (P_{42} \ll 2) + (P_{42} \ll 1) + (P_{43} \ll 2) + P_{44}$	$G = G_{sum} \gg 8$

Saturation & Output

The final output value of the 8-bit pixel is formatted by casting the resulting gradient to an 8-bit value. This truncates the result, meaning that in the cases of non-integers, the fractional bits are discarded without inspecting their value (round toward zero). If it is greater than 255 (the maximum possible pixel value), it is saturated to 255 (8'd255). The result is registered synchronously with clock edges through the valid_out signal to preserve synchronization.

4.2.4. IEEE 754 Floating-point Implementation

The implementation of the floating-point datapath requires multiple modules that interact with each other to form a multistep process that receives the input integers, converts them to IEEE float format, performs the desired calculations (addition/multiplication) and converts the result back to integer format to be casted to the output file, as described in Figure 4-7.

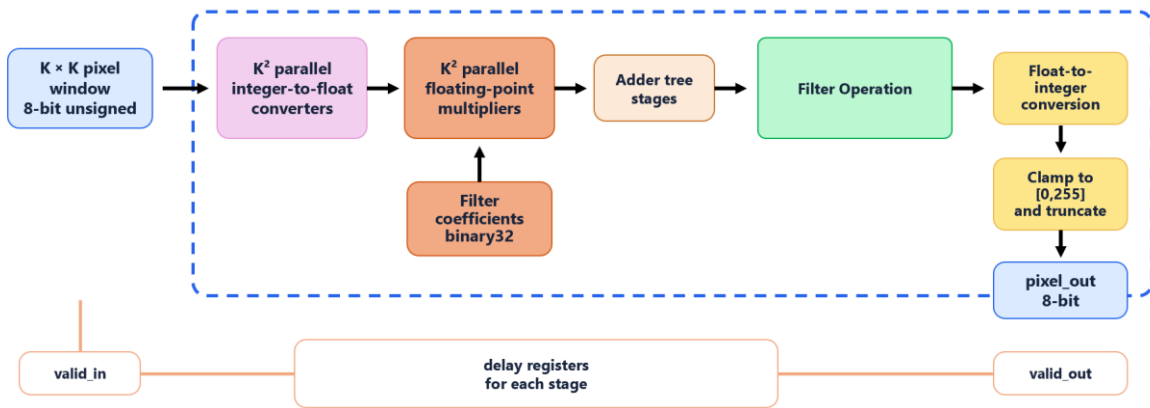


Figure 4-7: Architecture of the floating-point implementation.

The architecture used here follows the single-precision (binary32) format of the IEEE 754 standard, composed of a 1-bit sign (S), an 8-bit biased exponent (E), and a 23-bit mantissa or fractional (M).

$$A = (-1)^S \cdot (1.M) \cdot 2^{E-127} \quad [4-1]$$

Integer-to-Float Converter

This module converts the input 8-bit unsigned integer pixel value ($P \in [0,255]$) to a 32-bit floating-point number with zero cycle latency.

The sign bit is set as 0 ($S = 0$), since all values should be positive. The input is scanned to identify the position k of the Most Significant Bit ($k \in [0,7]$). The biased exponent is calculated as $E = 127 + k$. The mantissa is constructed from the remaining lower bits that are aligned left and padded with 0s to reach the 23-bit length. If the input pixel is zero ($P = 0$), the module returns an IEEE-compliant zero ($32'b0$).

Floating-Point Adder

The Adder module computes the addition between two IEEE float numbers in one clock cycle. First, the input float numbers are separated to their components: signs, exponents and expanded mantissas, which are

constructed to be 25-bits. Bit 23 restores the hidden leading bit (1.M), and bit 24 gives space for the arithmetic carry-out.

By comparing the exponents and finding the bigger one, the intermediate exponent part is defined. The mantissa corresponding to the smaller exponent is shifted right by $\Delta E = |E_a - E_b|$ positions. This aligns them and allows for proper comparison. If the input signs match, the mantissas are added and the sign is maintained. If the signs differ, the smaller mantissa is subtracted from the other one, and the sign is determined by the biggest. A 26-bit sum vector (sum_m[25:0]) keeps this result.

The sum vector is scanned for the position of the MSB. If an addition generates a carry-out (sum_m[25] = 1, sum_m[24] = 1), the exponent is increased by 2 or 1 respectively. For the remaining cases, the position of MSB is subtracted from the exponent, for example: if sum_m[13] = 1 then MSB is at the 10th place so $E = E_{\text{interm}} - 10$. The mantissa is constructed from the remaining lower bits that are aligned left and padded with 0s to reach the 23-bit length. Full cancellation of the numbers (sum_m = 0), results in an IEEE-compliant zero (32'b0).

Floating-Point Multiplier

The Multiplier module computes the product of two IEEE floats in one clock cycle. The input float numbers are separated into their components: signs, exponents and expanded mantissas, which are constructed to be 24-bits to restore the hidden leading bit (1.M).

The output sign is determined with a XOR gate from the input signs ($S = S_a \oplus S_b$), resulting to 0 (positive) if they are similar and 1 (negative) if they differ. The intermediate exponent is calculated as the addition of the exponents subtracting the bias $E_{\text{interm}} = E_a + E_b - 127$, store in a 9-bit vector to account for overflow. The mantissas are multiplied and stored as 48-bit intermediate product (prod_m[47:0]).

If the overflow bit of the product is filled (prod_m[47] = 1), the output exponent increases by one ($E = E_{\text{interm}} + 1$) and the following 23 bits of the product are kept as the output mantissa (prod_m[46:24]). Otherwise, the output is constructed from the calculated exponent and the first 23 bits of the product-mantissa without the overflow bits (prod_m[45:23]).

Float-to-Integer Converter

The final module converts the resulting floating-point value back to an 8-bit unsigned integer with zero cycle latency. The input float number is separated into its components: sign, exponent and expanded mantissa, which is constructed to be 24-bits to restore the hidden leading bit (1.M).

Since input pixel intensities are strictly positive, any negative values ($S = 1$) or exponent values that are smaller than 127 ($E < 127$, represent numbers smaller than 1, so the result is truncated to zero (8'd0). Exponent values equal to 135 or bigger ($E \geq 135$), correspond to numbers bigger than 256, so the value is saturated to 255 (8'd255), the maximum pixel value.

For the intermediate cases ($127 \leq E \leq 134$), the expanded mantissa is shifted right by $23 - (E - 127) = (150 - E)$ positions. The lower 8 bits (shifted_mant[7:0]) are the valid output pixel, which is truncated.

Core Filter Modules

The filter modules receive as input the window pixel values in 8-bit unsigned integer form and convert them to IEEE 754 floating-point values by using instances of the Integer-to-Float module, with zero cycle latency. Then, the resulting values are multiplied with the filter weights in IEEE 754 float format (e.g. $1 \equiv 32'h3f800000$) through the Float-Multiplier module, in one clock cycle.

The following step is to add all the products, but since the Float-Adder module receives only two values as input to avoid critical delays, this process is broken down to multiple steps in an Adder Tree format, as shown in Figure 4-8.

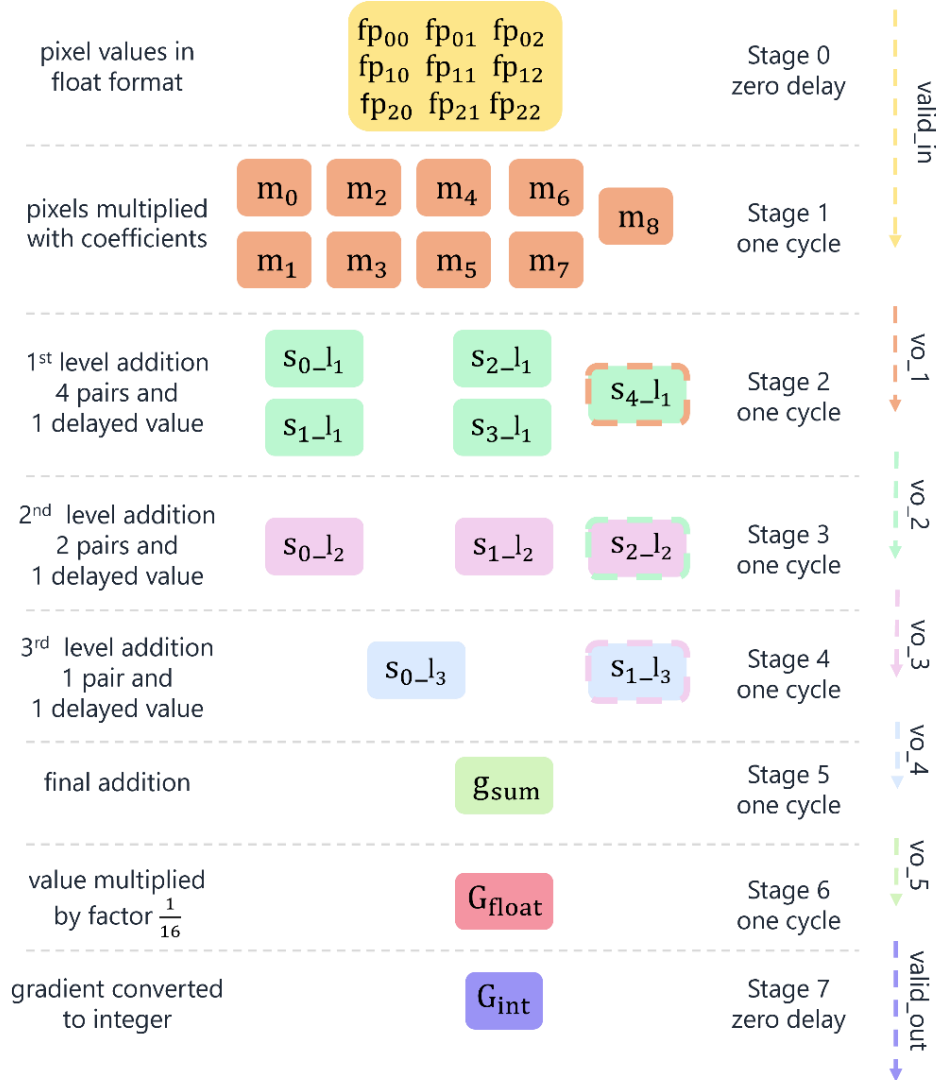


Figure 4-8: Adder tree stages and propagation of valid signal for the 3x3 Gauss core filter module.

In each stage (1 cycle) the values are added in pairs to create partial sums. In each reduction where an odd operand remains unpaired, a synchronous register delays the value by exactly one clock cycle, so it remains available to be added in the following stages. The process is repeated until the final addition and calculation of the gradient. The Float-to-Integer module is used to convert the final gradient value to an 8-bit integer. Each stage receives a “valid_in” and sends a “valid_out” signal, creating a stage-by-stage “handshake” chain (valid_in → vo_1 → vo_2 → → valid_out) to maintain the proper flow in the arithmetic execution.

4.2.5. Cascaded Filters

The implementation of cascaded filters is almost identical to the stand-alone cases, with some adjustments. The floating-point filter cases propagate the output value from the Gaussian blur as a float, without clipping

and converting to integer. The value passes through the second line buffer that handles only float values and is inserted as input in the second filter module. The Sobel filter executes the edge detection calculations, truncates the value and converts it to an 8-bit binary integer. In the equivalent fixed-point implementation, the value is clipped to 8-bits after the first filter and passed to the second line buffer.

Both in floating-point but also fixed-point implementations, the final Sobel magnitude is scaled before it is cast to the result. The purpose is to readjust the range of values, to avoid most of them being clipped to the maximum value (255) and create more evident edges. Each value is divided by $\frac{\max(|G_x|+|G_y|)}{255}$, where the maximum sum is calculated from the target image values, so division by 8 for the 3×3 kernel and by 64 for the 5×5 . For fixed-point arithmetic, division is performed by a right shift of 3 or 6 positions, and for the floating-point implementations by multiplying with 0.125 and 0.015625 respectively.

4.2.6. Testbench

The Testbench module is used in the behavioral simulation to read the input, create the final output and measure the cycle count and latency. The text file of the preprocessed image is read line-by-line and pixel values are injected on each positive clock edge (`@(posedge clk)`), while activating the `valid_in` signal, to simulate a continuous pixel stream. The signal and pixel value are driven to the Device Under Test (DUT) which is an instance of the Top module. When the end of the file is reached, the signal is deactivated.

In a continuous loop, at every positive clock edge and when the `valid_out` signal is activated, filtered pixels (`pixel_out`) are written to the output text file. When the expected number of pixels, which is defined by the image dimensions, is written to the file, the loop breaks.

The Testbench module also has a global cycle counter for the total simulation cycles, and measures latency by keeping the exact cycle at which `valid_out` activates for the first time, meaning when the first pixel is seen.

4.3. Post-processing

4.3.1. Custom Filters

To establish an honest baseline for evaluating the synthesized hardware outputs, the software reference model must match the hardware. This implies the need to match the hardware in various factors: kernel coefficient values, boundary condition handling, intermediate precision accumulation, saturation logic. While there are multiple libraries that provide various filters implemented with different techniques, to ensure the needed similarity, custom Python reference implementations were developed.

The correctness of these custom filters was cross-checked against standard OpenCV equivalents and scored highly. Custom implemented filters are still needed to be able to replicate the fixed-point truncation and saturation of the hardware.

It is worth mentioning that convolution in Python for non-integer filters is performed using float32 format, and the result is truncated and clipped, to match the functionality of the RTL implementations. Also, for the Gaussian filter, each architecture was compared to an equivalent Python implementation to ensure the correctness of the design, meaning that the integer datapath was compared to a Gauss implementation with integer coefficients. But, for the final comparison, the implementation with the exact coefficients was used to measure the accuracy of the design.

4.3.2. Image Reconstruction

The output file generated by the behavioral simulation testbench is the input to the post-processing module. Each 8-bit decimal intensity value is appended into a flat 1D array. The extra padding is eliminated during the RTL calculations, so the output pixel stream corresponds exactly to the source image resolution ($W \times H$).

Using NumPy and OpenCV (cv2.imwrite) the 1D array is reshaped into an 8-bit unsigned integer 2D matrix (dtype = uint8) matching the target dimensions and then saved as an image.

4.3.3. Image Comparison

The similarity of the reconstructed hardware images compared to the software reference is evaluated using standard image quality assessment metrics, that are also mentioned in 2.5.1: Maximum Absolute Difference, Mean Squared Error and Root Mean Squared Error, Peak Signal-to-Noise Ratio, Structural Similarity Index (computed using the skimage.metrics library)

As an addition to these metrics, two visualization plots are created using the matplotlib library:

1. **SSIM Quality Maps** (cmap='gray'): Highlights localized regions that have structural differences
2. **Absolute Difference Heatmaps** (cmap='hot'): Visual mapping of pixel offsets. Provides information for edge misalignment, or underflow/overflow cases.

4.3.4. Hardware Performance and Resource Metrics

To assess the hardware trade-offs between fixed-point and custom binary32 floating-point arithmetic, multiple factors, were tracked across all different cases. Implementation and Synthesis were run on the designed hardware, with a timing constraint file, through the Vivado Design Suite and two reports were extracted.

1. **Utilization Report:** includes information about the physical FPGA footprint on the target device, with metrics such as LUTs, Flip-Flops, DSPs and BRAMs
2. **Report Timing Summary:** provides the timing analysis under a constrained target period T_{target} through factors such as WNS.

The Behavioral Simulation provides information about the latency and total cycles. Also, by design, the hardware operates one pixel per cycle. Collectively, this information allows for the calculation of other important metrics, analyzed in section 2.5.2.

CHAPTER 5: Results

This section presents the results of all hardware filter architectures developed in this thesis, focusing on the image quality and the resources spent. The main objective is to evaluate the trade-offs between the resource-efficient integer shift-and-add architectures and the custom IEEE 754 single-precision floating-point datapaths for different window sizes.

The test image used is of the author's cat, named "Smeagol", with an initial resolution of 364x483 pixels. It features sharp edges, high-contrast, and different textures (e.g. leaves, cat's body). These characteristics make it suitable for spatial filtering assessment, as such details can expose arithmetic and rounding differences. The same image was used in all hardware designs to maintain a constant input signal, ensuring a fair and consistent comparative evaluation.

All hardware architectures were designed, synthesized and implemented in AMD Xilinx Vivado Design Suite 2025.2 using the SystemVerilog language. The target clock period was set to $T_{\text{target}} = 20\text{ns}$ (50MHz) for all cases.

Initially, the board AMD Kintex-7 FPGA (xc7k70tfbv676-1) was used, with 41000 LUTs, 82000 FFs, 240 DSPs, but it did not have sufficient resources to support the high LUT demand of some architectures. To maintain a consistent basis of comparison across all evaluated filters, the target device was updated to the AMD Artix-7 FPGA (xc7a200tsbg484-1), which contains 134600 LUTs, 269200 FFs and 740 DSPs.

The preprocessing and post-processing stages were developed in Python 3.13.0, using the following libraries: *cv2* for reading and writing images, *numpy* for data manipulation and *matplotlib* and *plotly* for plotting the results. RTL designs, pre and post processing code, reports and output files of the experiments are available in Github: <https://github.com/NefeliZ/thesis-image-analysis-fpga>

5.1. Preprocessing

The original image is imported in the preprocessing script, converted to 8-bit grayscale with extra padding and the pixel values are exported as a text file. Figure 5-1 is a representation of the output text file that contains the binary pixel values that will serve as the input stream for the following stages.

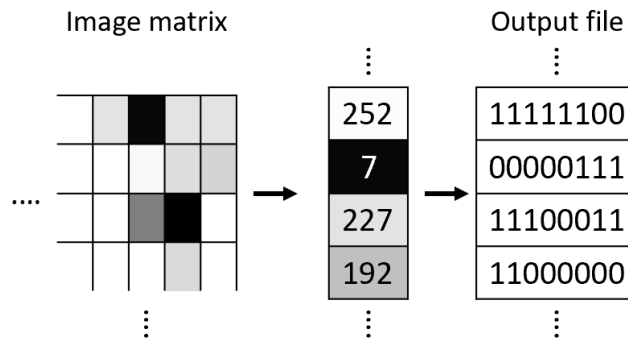


Figure 5-1: Schematic representation of the image pixel values converted to a sequence of binary values in the text file.

The padding width of the image is determined by the kernel size and the filter case. The dimensions of the created padded images were checked and have the expected size as described in Section 4.1.1. Table 5-1 shows the resulting dimensions for each case, and Figure 5-2 shows the original image (364x483) and the gray padded image (372x491) prepared for the 5x5 cascaded filter case.

Table 5-1: Dimensions of padded images for each filter case, with initial image size (364x483).

Filter Case	Formula	Result
3x3 Filter	$(W + 2) \times (H + 2)$	(366×485)
3x3 Cascaded Filters	$(W + 4) \times (H + 4)$	(368×487)
5x5 Filter	$(W + 4) \times (H + 4)$	(368×487)
5x5 Cascaded Filters	$(W + 8) \times (H + 8)$	(372×491)

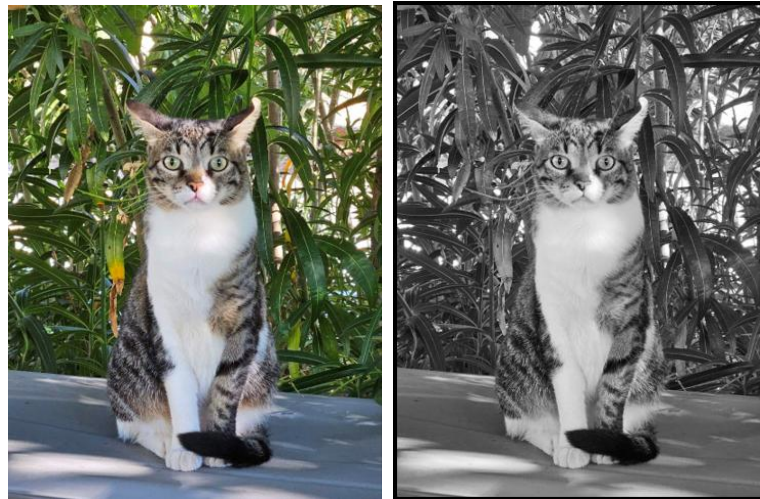


Figure 5-2: The original RGB image (left) and the 8-pixel padded grayscale transformation (right).

5.2. Hardware Synthesis and Implementation

Hardware resource and timing metrics were collected for all 12 different filter implementations and were used to understand the efficiency of each design. All values are presented in Table 5-2.

Table 5-2: Hardware usage and timing metrics for all filter architectures.

Filter Case	Arithmetic	LUTs	FFs	DSPs	WNS (ns)	Latency Cycles	Total cycles
Sobel 3x3	Integer fixed-point	2340	6092	0	5.101	737	177512
	IEEE floating-point	10025	6716	14	0.789	741	177516
Gaussian 3x3	Integer fixed-point	2316	6093	0	7.045	737	177512
	IEEE floating-point	7864	6666	18	4.634	741	177516
Cascaded 3x3	Integer fixed-point	4721	12233	0	6.819	1481	179220
	IEEE floating-point	23008	30705	34	1.07	1490	179229
Sobel 5x5	Integer fixed-point	4677	12173	0	1.389	1479	179218
	IEEE floating-point	32232	14556	66	0.404	1485	179224
Gaussian 5x5	Integer fixed-point	4544	12173	0	5.508	1479	179218
	IEEE floating-point	21200	13738	50	3.974	1484	179223
Cascaded 5x5	Integer fixed-point	9028	24482	0	2.561	2989	182656
	IEEE floating-point	59267	64667	118	-1.27	3001	182668

This information was used to calculate other metrics such as maximum operating frequency, throughput, FPS, the percentage of the total board resources that are used, and area efficiency as throughput per 1000 LUTs. Based on equation [2-26] and the fact that the designs produce one pixel per cycle, throughput and maximum operating frequency are identical. The values are presented in Table 5-3.

Table 5-3: Calculated usage and timing metrics for all filter architectures.

Filter Case	Arithmetic	F _{max} (MHz)	FPS	% LUT	% FF	% DSP	Area Efficiency (T/kLUT)
		T (10 ⁶ p/s)					
Sobel 3x3	Integer fixed-point	67.1186	378.1074	1.738484	2.263001	0	28.68316
	IEEE floating-point	52.05351	293.2328	7.447994	2.494799	1.891892	5.19237
Gaussian 3x3	Integer fixed-point	77.19027	434.8454	1.720654	2.263373	0	33.32913
	IEEE floating-point	65.07875	366.6078	5.842496	2.476226	2.432432	8.275527
Cascaded 3x3	Integer fixed-point	75.86678	423.3165	3.507429	4.544205	0	16.07007
	IEEE floating-point	52.8262	294.7414	17.09361	11.40602	4.594595	2.295993
Sobel 5x5	Integer fixed-point	53.73166	299.8118	3.47474	4.521917	0	11.48849
	IEEE floating-point	51.03082	284.7321	23.94651	5.407132	8.918919	1.583235
Gaussian 5x5	Integer fixed-point	69.00359	385.026	3.375929	4.521917	0	15.18565
	IEEE floating-point	62.3986	348.1618	15.75037	5.103269	6.756757	2.94333
Cascaded 5x5	Integer fixed-point	57.34274	313.9384	6.707281	9.094354	0	6.351655
	IEEE floating-point	47.01457	257.3772	44.03195	24.02192	15.94595	0.793267

5.2.1. Hardware Resource Metrics

Figure 5-3 shows the percentage of the total board resources used in each architecture. Integer fixed-point implementations do not use any DSPs, and their overall logic usage is smaller compared to the IEEE 754 equivalent. Resources also scale with window size in all cases, especially in the floating-point architectures.

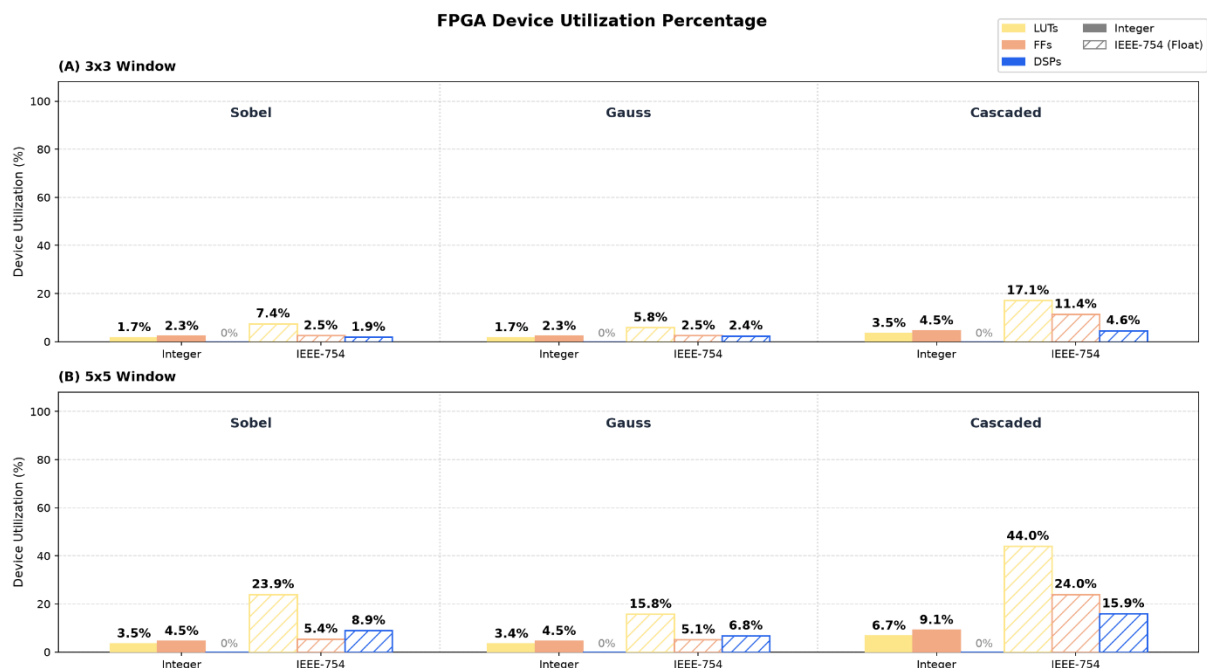


Figure 5-3: Percentage of total board hardware resources used by kernel size and architecture.

The device views of every architecture created at the Vivado implementation stage are presented below for each filter case [Figure 5-4, Figure 5-5, Figure 5-6]. They depict how the logic slices are placed on the clock regions of the target FPGA. As indicated by the rest of the figures, more complex pipelines and larger kernels need more resources and consequently occupy more area on the FPGA board.

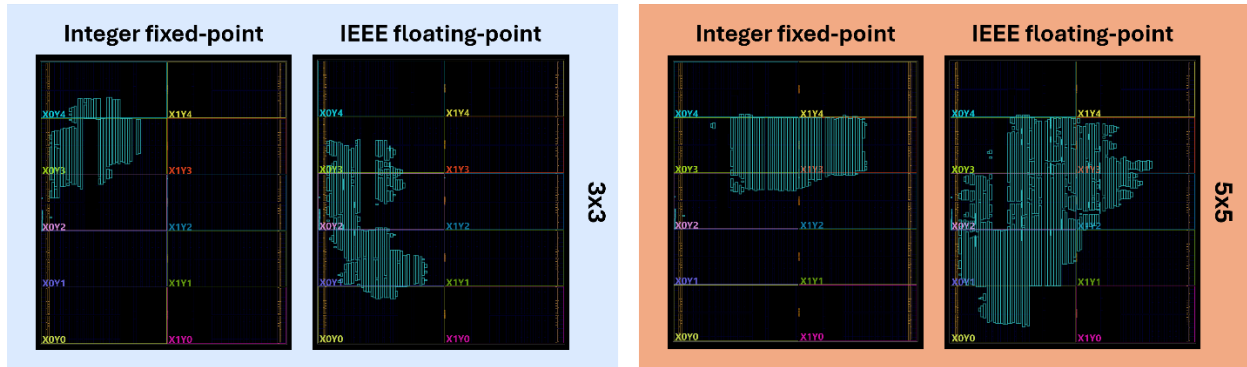


Figure 5-4: Post-implementation device view of Sobel filter architectures.

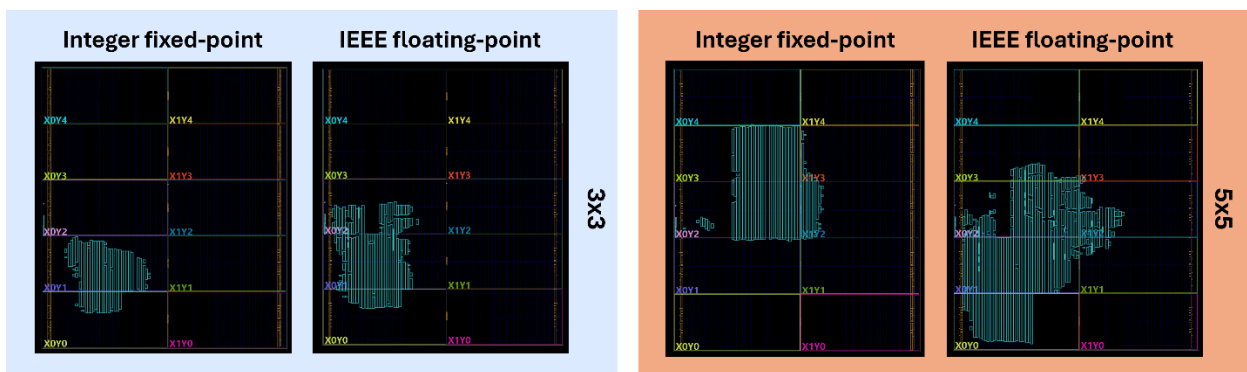


Figure 5-5: Post-implementation device view of Gauss filter architectures.

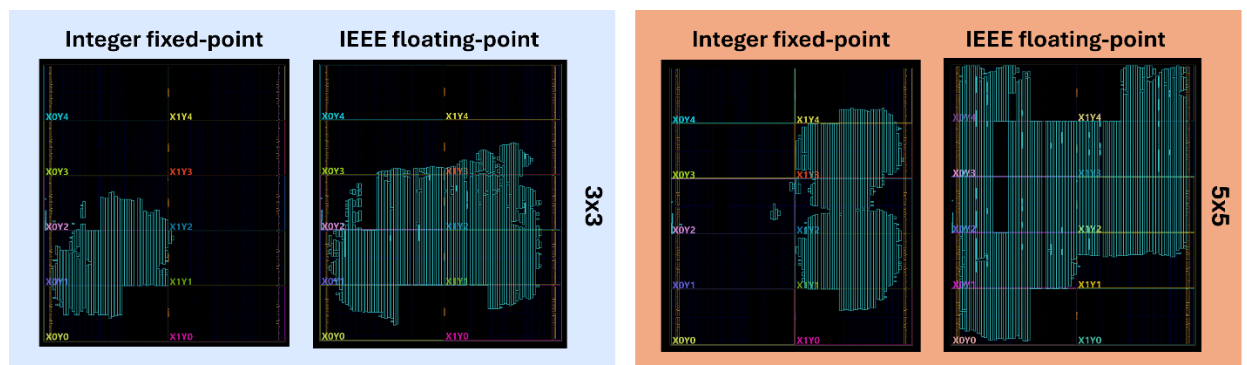


Figure 5-6: Post-implementation device view of Cascaded filters architectures.

The efficiency of the area occupied on a FPGA board is calculated as the ratio of throughput to hardware resource usage. Figure 5-7 shows the efficiency of each architecture. There is a significant difference between integer and floating-point implementations in all filter cases, with the first one performing significantly better.

Even though operating frequency and throughput are comparable across architectures, when considering the number of resources used, area efficiency is significantly reduced.

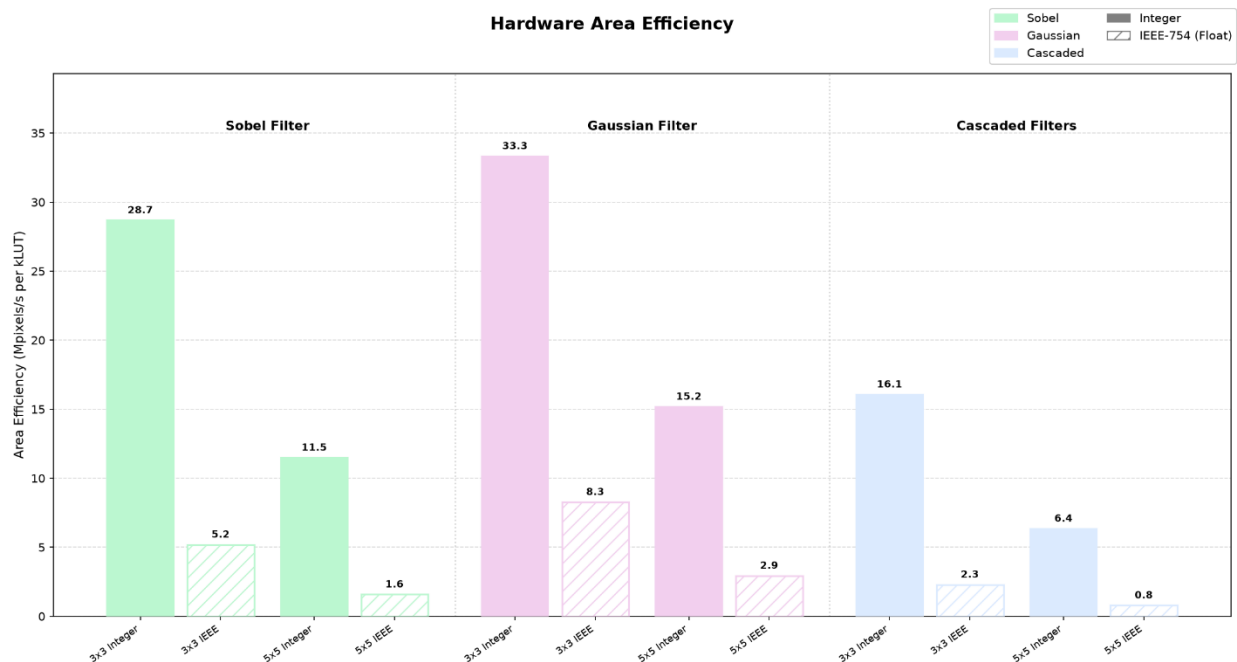


Figure 5-7: Hardware area efficiency calculated as achieved throughput per kLUT used, for all cases.

Figure 5-8 shows the Root Mean Squared Error (RMSE) achieved in respect to the total LUTs used, correlating the quality of the output image with the hardware resources spent to achieve it. The floating-point arithmetic generally produces an output with smaller error compared to the equivalent fixed-point implementation, but at a higher cost that scales with kernel size and pipeline complexity.

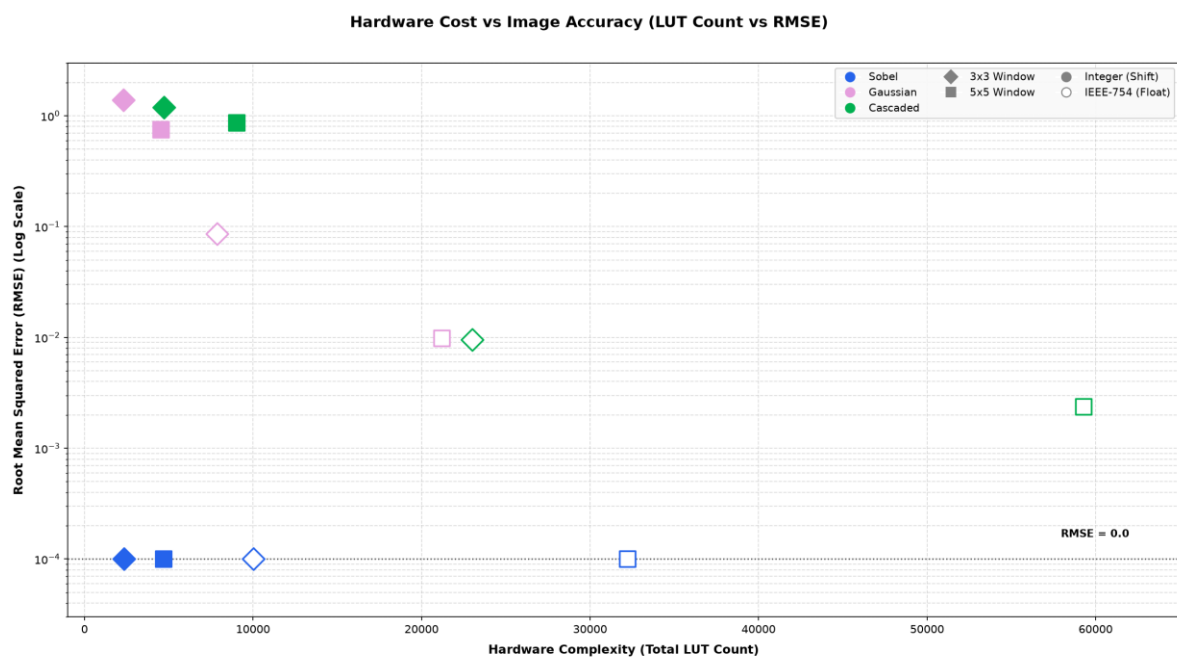


Figure 5-8: Total LUTs used and the respective RMSE achieved for each case.

5.2.2. Timing Analysis, Execution Cycles and FPS

The target clock period was set at $T_{\text{target}} = 20\text{ns}$, consequently the target frequency is $F_{\text{target}} = 50\text{MHz}$. For the designs with a positive Worst Negative Slack (WNS), the maximum operating frequency, calculated using equation [2-3], is higher than the target frequency. In the Cascaded filters 5x5 floating-point implementation, WNS is negative and F_{max} is lower than the target frequency. Figure 5-9 presents these numbers for each architecture.



Figure 5-9: Maximum Operating Frequency and Worst Negative Slack for each case.

Figure 5-10 shows the initial latency and the total execution cycles of each implementation. They indicate the pipeline warm-up duration before the first valid output pixel is produced and the total time required to process an entire frame. Figure 5-11 shows the achieved frame rate, as calculated by the equation [2-4].

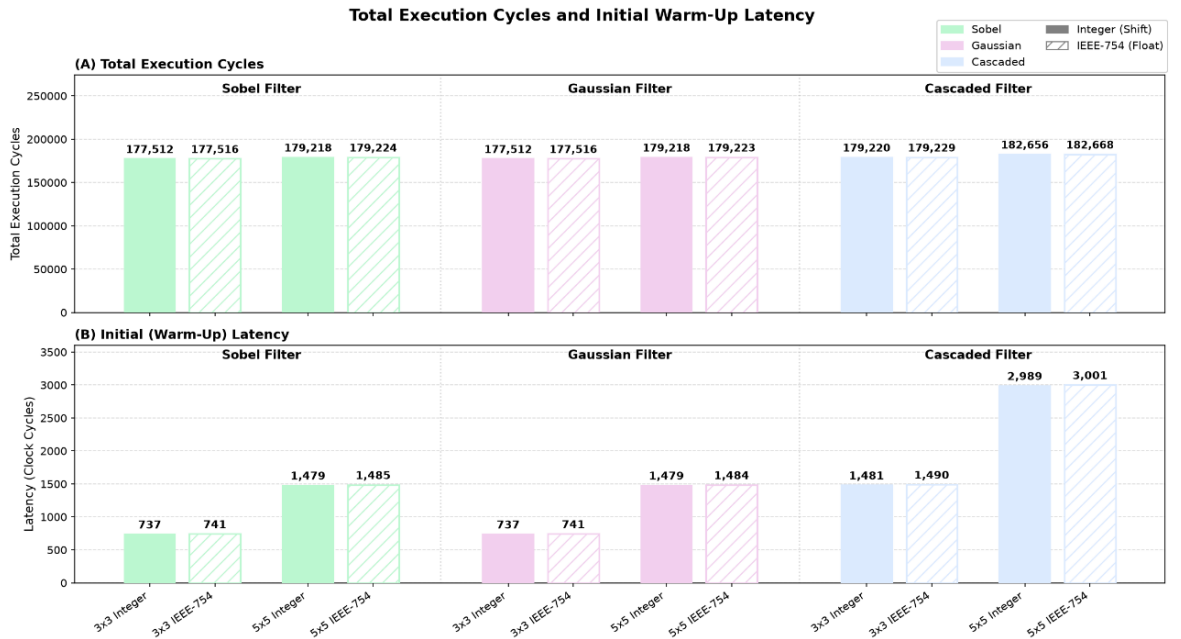


Figure 5-10: Total operating clock cycles and initial latency for each filter architecture.

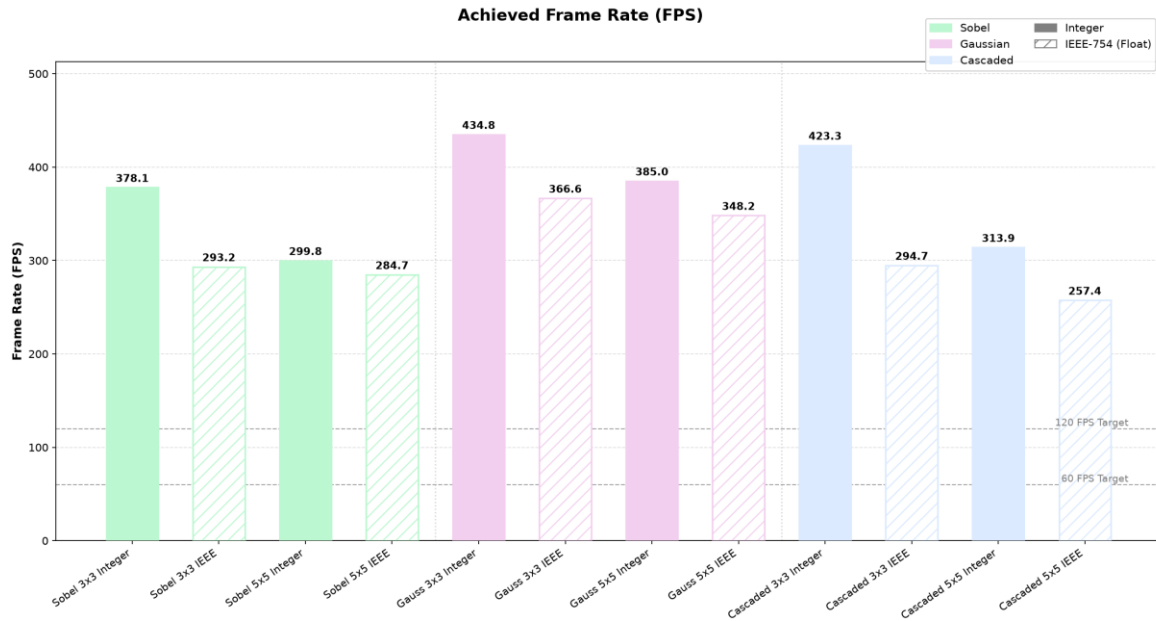


Figure 5-11: Achieved Frame Rate for all cases.

5.3. Post processing and Quality verification

5.3.1. Software Baseline Verification

The custom implementations of the filters in Python were cross-checked against standard OpenCV equivalents, for both kernel sizes and the combination of cascaded filters. The resulting images were identical in every case, proving the correctness of the custom filters. The metrics of the comparison, that are analyzed in section [2.5.1], are presented in Table 5-4, alongside the SSIM plot and absolute difference heat map in Figure 5-12, which are of little interest since they have no information. Figure 5-13 shows the two filtered images for the 3x3 Sobel edge detection implementation.

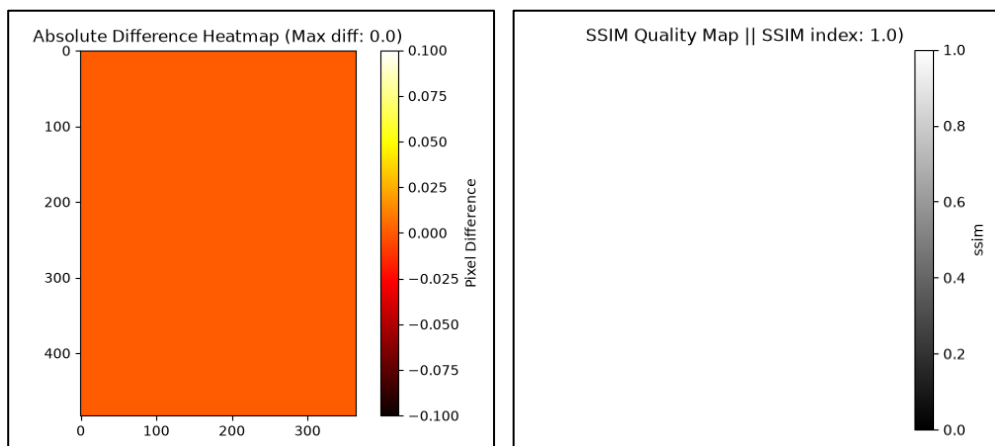


Figure 5-12: Absolute difference heat map and SSIM plot of custom 3x3 Sobel filter implementation and OpenCV equivalent.

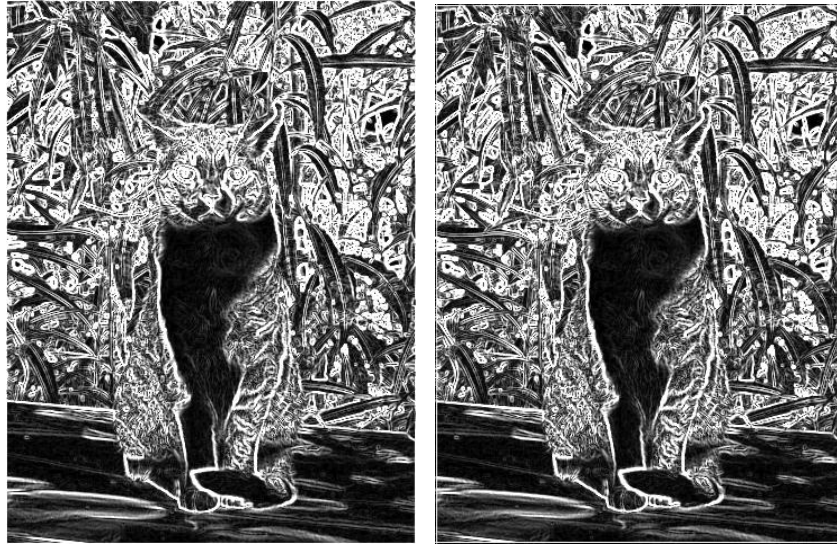


Figure 5-13: Comparison of custom 3x3 Sobel filter (left) and OpenCV equivalent (right).

Table 5-4: Image quality metrics for custom filter implementation in Python.

Metric	Value
MAD	0.0
MSE	0.0
RMSE	0.0
PSNR (dB)	∞
SSIM	1.0

5.3.2. Hardware Image Reconstruction

The practical validity of the hardware filters was evaluated through the serialized output streams generated by the simulations. They were reconstructed into 2D images with equal dimensions to the original image (364x483), and the images were inspected visually to ensure there were no pixel drifts, or off-by-one errors. Figure 5-14, Figure 5-15 and Figure 5-16 show the reconstructed images for the Sobel filter, the Gaussian blur and the cascade of filters respectively, for 3x3 and 5x5 kernel sizes and different applied arithmetic.

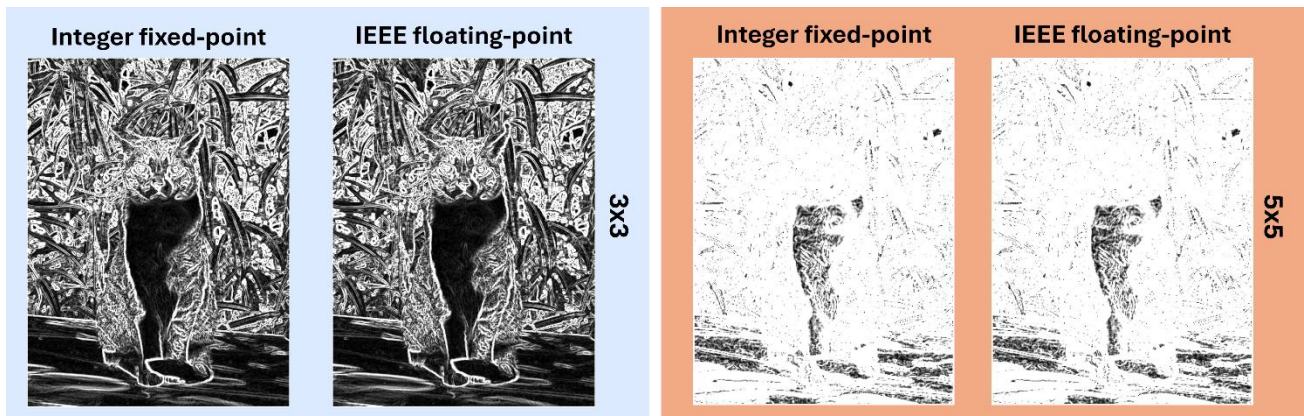


Figure 5-14: Reconstructed images for Sobel filter.

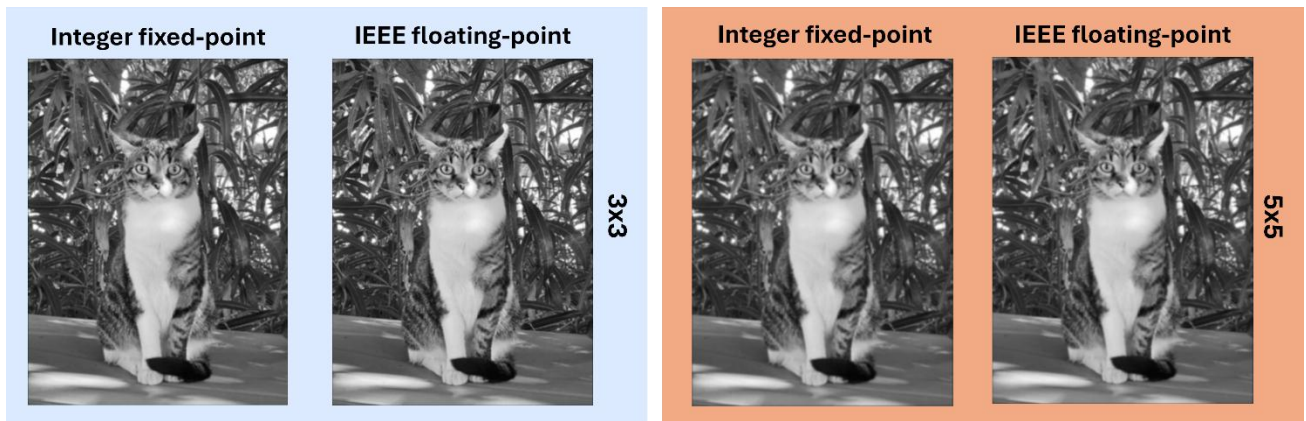


Figure 5-15: Reconstructed images for Gaussian filter.

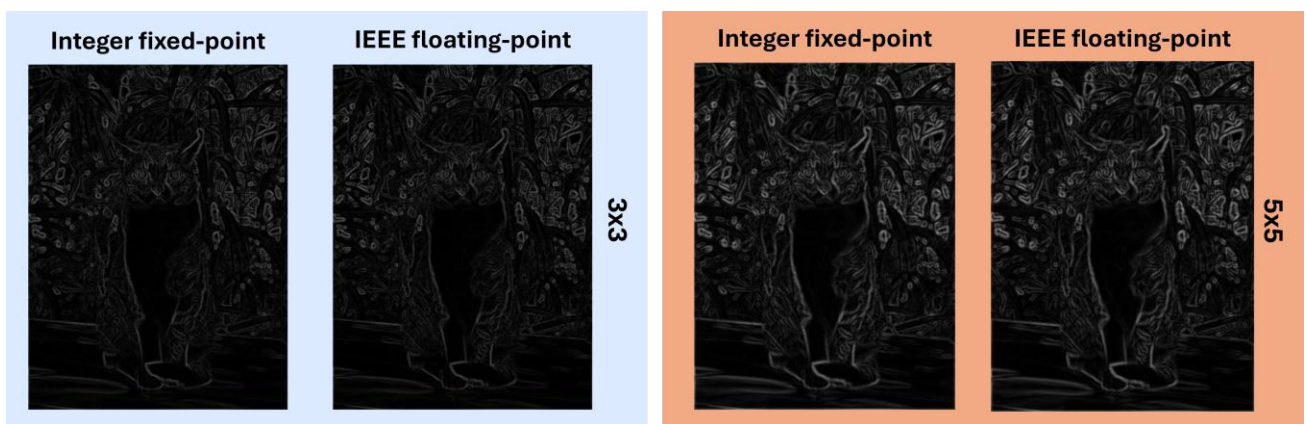


Figure 5-16: Reconstructed images for cascaded filters.

5.3.3. Image Comparison

Using the image quality metrics analysed in section [2.5.1] all reconstructed images were compared with the respective reference image. The results for 3x3 kernel are presented in Table 5-5, and for 5x5 in Table 5-6.

Table 5-5: Image comparison metrics for all filter cases with 3x3 window.

Filter Case	Arithmetic	MAD	MSE	RMSE	SSIM	PSNR
Sobel	Integer fixed-point	0	0	0	1	∞
	IEEE floating-point	0	0	0	1	∞
Gaussian	Integer fixed-point	14	1.949321	1.396181	0.99741	45.23197
	IEEE floating-point	1	0.007485	0.086517	0.999913	69.38873
Cascaded	Integer fixed-point	9	1.425739	1.194043	0.994982	46.5904
	IEEE floating-point	1	0.000091	0.00954	0.999998	88.54009

Table 5-6: Image comparison metrics for all filter cases with 5x5 window.

Filter Case	Arithmetic	MAD	MSE	RMSE	SSIM	PSNR
Sobel	Integer fixed-point	0	0	0	1	∞
	IEEE floating-point	0	0	0	1	∞
Gaussian	Integer fixed-point	5	0.565974	0.752312	0.998973	50.60284
	IEEE floating-point	1	0.0000967	0.009833	0.999999	88.2768
Cascaded	Integer fixed-point	6	0.757042	0.870081	0.998019	49.33961
	IEEE floating-point	1	0.0000056	0.002385	0.999999	100.5813

As derived from the values in the tables above, Sobel filter implementations in all cases were identical to the reference image. Consequently, the SSIM plot and absolute difference heat map, do not show any differences and are identical across all Sobel filter cases (Figure 5-17).

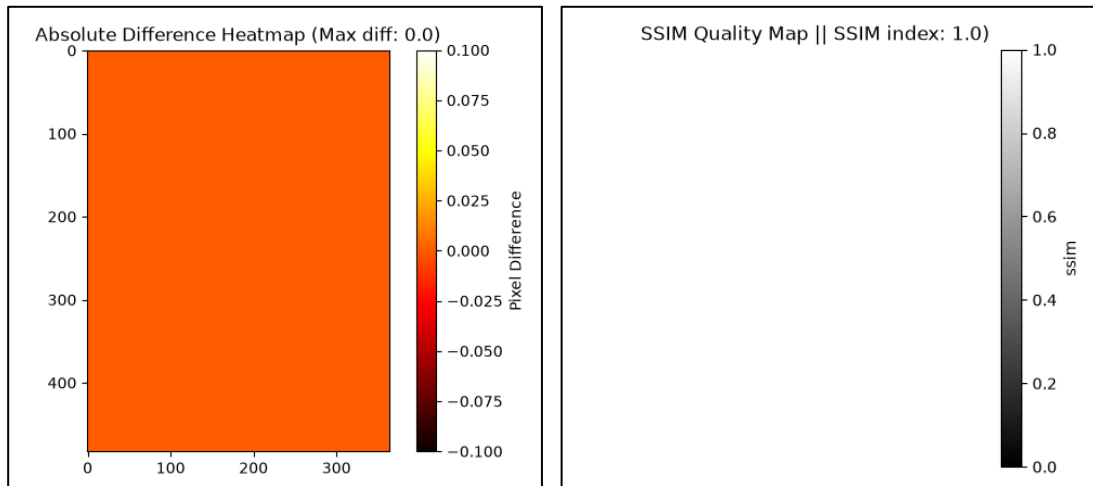


Figure 5-17: Absolute difference heat map and SSIM plot of all Sobel filter cases and their respective reference.

In the case of the Gaussian filters, the SSIM plots show no actual structural differences, and they are not presented here. The absolute difference heat maps indicate value divergence from the reference image that is minimal small errors (± 1) for the floating-point design, but much more evident in the integer implementations, highlighting the accuracy of the first. The plots are presented in Figure 5-18 and Figure 5-19 for both kernel sizes.

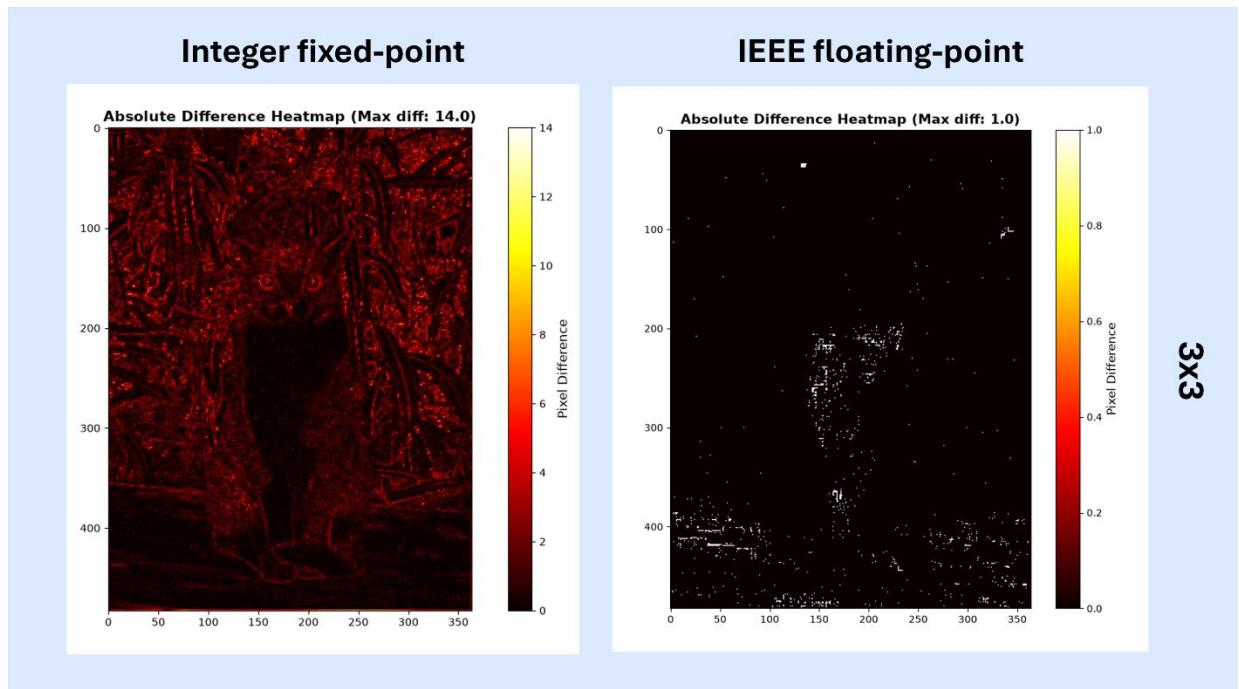


Figure 5-18: Absolute difference heat map of integer and floating-point 3x3 Gaussian filter implementations.

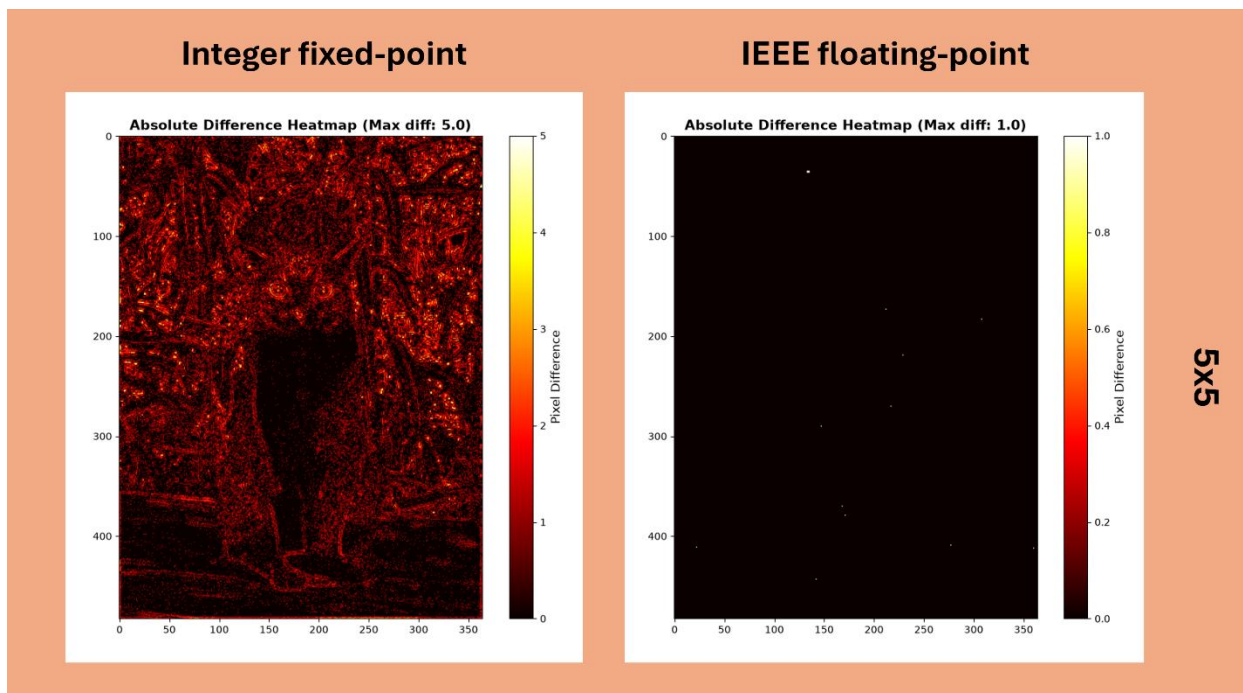


Figure 5-19: Absolute difference heat map of integer and floating-point 5x5 Gaussian filter implementations.

For the cascaded filters, the SSIM plots show no structural differences as well. However, there are differences between the reconstructed and the reference images regarding the pixel values. Both fixed-point designs show medium value differences, while the floating-point implementations have a very limited number of pixels with ± 1 difference. They are displayed as small white specs in the plot. Figure 5-20 and Figure 5-21 show the heat map plots for kernels 3x3 and 5x5.

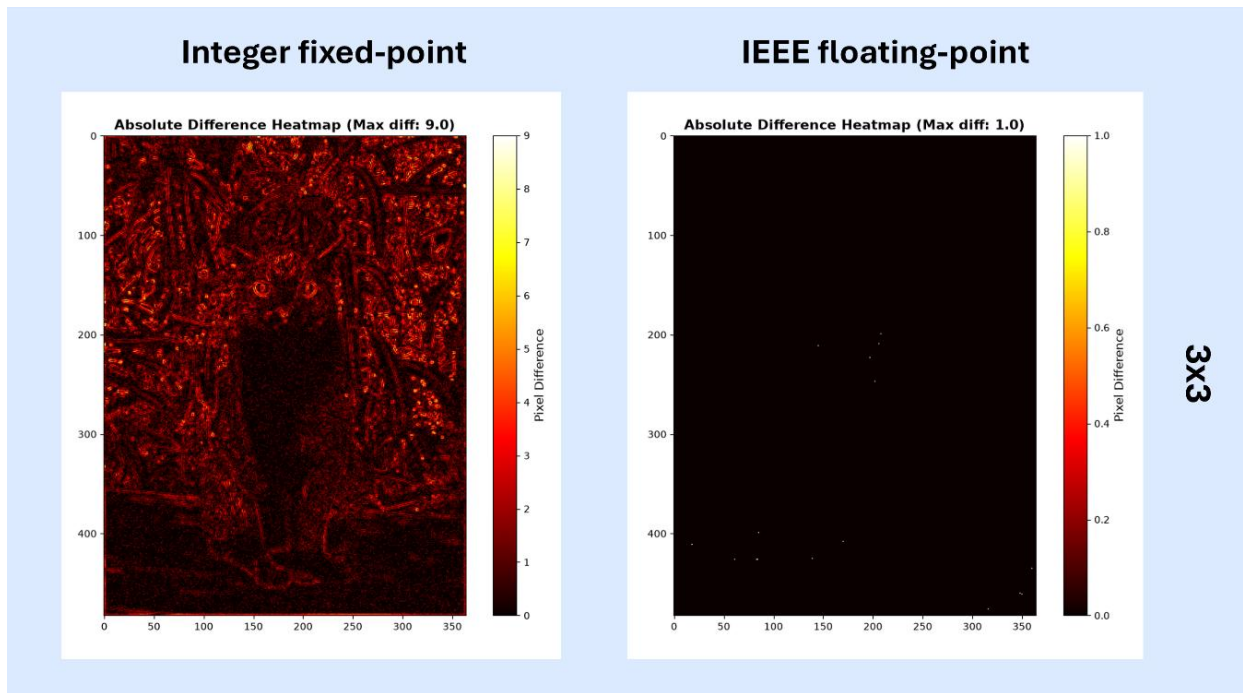


Figure 5-20: Absolute difference heat map of integer and floating-point 3x3 cascaded filters implementations.

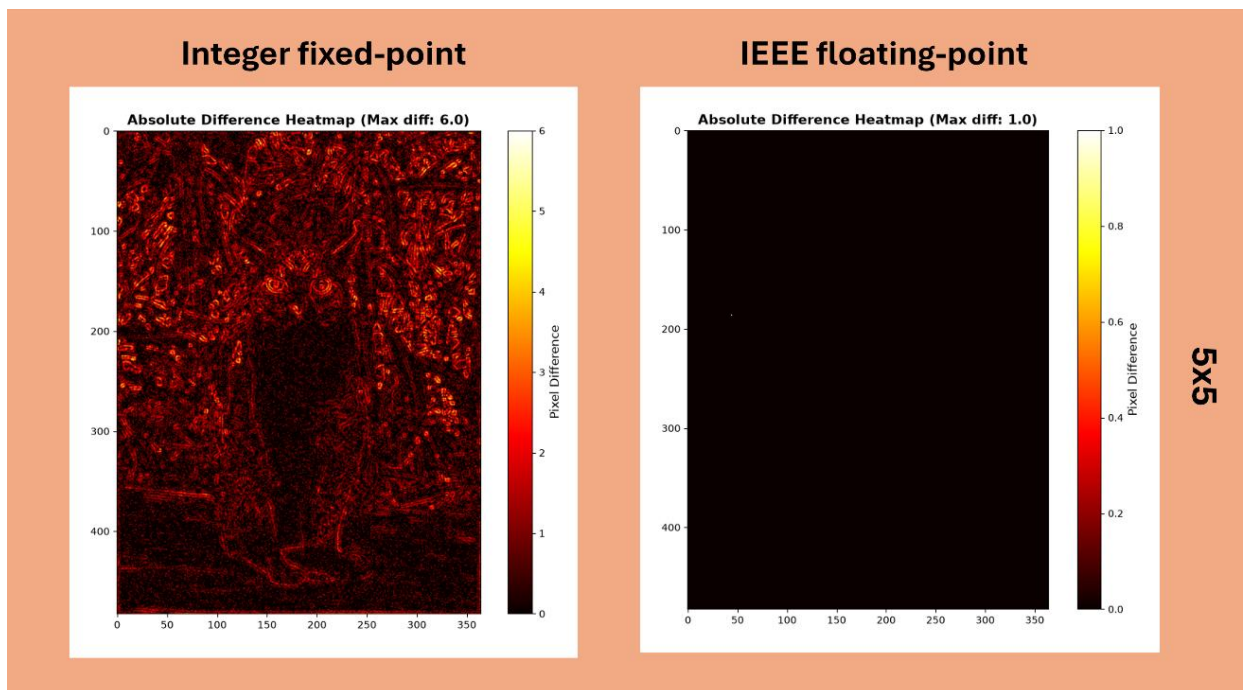


Figure 5-21: Absolute difference heat map of integer and floating-point 5x5 cascaded filters implementations.

CHAPTER 6: Discussion

6.1. Major findings

This thesis compares twelve FPGA implementations of Sobel, Gaussian, and cascaded Gaussian-Sobel spatial filters. Each filter was implemented with 3x3 and 5x5 kernels using either multiplier-less integer arithmetic or a custom binary32-format floating-point datapath. The comparison considered hardware-resource consumption, timing, latency, area efficiency, and agreement with software reference models. The results of each implementation are discussed in this section.

6.1.1. Hardware Resource Metrics and Timing Analysis

Every floating-point architecture consumed more hardware resources than its corresponding fixed-point implementation. These modules make use of DSP blocks, which are not used at all in the shift-and-add arithmetic, but also LUTs and FFs are increased. A notable example is the 3x3 cascaded filters case, where LUT usage increased from 3.5% to 17.1% of the total available slices when changing numerical precision.

Moreover, the hardware resources used scale similarly with the kernel size across all implementations. Also, it is evident from the post-implementation device views that more complex pipelines and larger kernels occupy a larger area on the FPGA board, but this is not measurable solely from these images. The calculated throughput for each design along with the total number of LUTs used, allows for the calculation of the hardware area efficiency. This metric indicates the collective effectiveness of an architecture. Integer arithmetics and smaller kernels have better performance, with the Sobel 3x3 and Gauss 3x3 achieving the highest efficiency as shown in Figure 5-7.

Almost all filter cases achieve a positive Worst Negative Slack (WNS), which means that there is room for optimization that will increase the maximum operating frequency. In some cases the margin is small, while the 3x3 fixed-point Gaussian blur there is a headroom of 7.04ns, which results to an increased frequency of 77.2MHz, compared to the target 50MHz. The 5x5 cascaded floating-point architecture does not have timing closure and returns a WNS = -1.27ns for the target period $T_{\text{target}} = 20\text{ns}$ and consequently a maximum frequency of $F_{\text{max}} = 47.0\text{MHz}$ smaller than 50MHz. The period should be increased at least by 2ns to achieve timing closure and ensure the designs's reliability.

The latency cycles and total execution cycles presented in Figure 5-10, are almost equal between different arithmetics for the same kernel size. Sobel and Gauss implementations differ by 4 cycles and cascaded differ by 9. This is expected because of the architecture of the core filter modules, line buffers and number of input pixels. Even though these values were calculated through the behavioral simulation, latency and operational cycles can be computed at the design-time and are fixed and independent of other parameters.

The achieved frame rate (FPS) scales inversly with kernel size and pipeline complexity, with the highest 434.8 frames/s by 3x3 Gauss fixed-point and the lowest 257.4 frames/s by 5x5 cascaded floating-point. Every design though yields a FPS higher than some industry standars such as real-time video at 60 FPS or high-rate displays at 120 FPS.

6.1.2. Image Quality

In the case of the Sobel filter, the kernels have only integer coefficients ($\pm 1, \pm 2, 0$) and any scaling, normalization or division that occurs is implemented with power-of-two shifts. Because this shift is applied

identically in the hardware and in the Python reference, both the integer and the floating-point datapaths compute the result bit-for-bit exactly, for both kernel sizes. This results in images with 100% similarity to the reference in every case, since the same integer arithmetic is performed in Python filter implementations. The only differentiation is the hardware resources spent.

In the case of the Gaussian blur, the kernel coefficients are fractional. The integer datapath approximates the kernel with integer coefficients and normalizes the weighted sum with a right shift by a power-of-two, while the floating-point datapath uses the exact coefficients in float format. The reference image is computed with the exact gaussian values as well. Consequently, the two implementations produce similar results, as seen in Figure 5-15, but the later differs from the reference only by a few pixels, while the integer result carries error from the coefficient approximation and the truncation of the fractional bits during the shift. This creates evident differences in areas where intensity changes, such as areas with edges and texture, that are highlighted in the fixed-point heat maps of Figure 5-18 and Figure 5-19 (the leaves, the eyes, the outline). The number of offset pixels seems to decrease with a larger kernel in this case.

The cascaded filter pipeline operates in a similar way. In the floating-point design the intermediate Gaussian result is passed to the Sobel stage as a float and is converted to an integer only at the final stage, so no precision is lost between the two filters. Compared with the Python reference, the floating-point cascade gives nearly identical results, with only a few pixels differing by a value of ± 1 , because of rounding in the floating-point additions. The integer cascades showed larger differences, and in a wider range of the image, because the intermediate output is truncated to an 8-bit integer before entering the second filter stage. These errors appear mainly on edges and in textured regions, where the Sobel operator amplifies the quantization error of the Gaussian stage, which confirms that errors accumulate along a cascaded integer pipeline.

6.1.3. Quality compared to resource cost

While yielding the same accuracy, the 5x5 Sobel integer uses 4677 LUTs, and the floating-point equivalent occupies 32232 LUTs, while also having a reduced frame rate and maximum operating frequency. Consequently, the floating-point architecture does not offer any advantage in filters with integer coefficients, as expected. Despite the increased accuracy, the 3x3 floating-point Gaussian blur used 7864 LUTs, 6666 FFs and 18 DSPs, against 2316, 6093 and 0 for the integer case. The hardware area efficiency is decreased by 4 times when moving between those architectures. The higher accuracy of the IEEE 754 cascades also has a large hardware cost. The second line buffer stores 32-bit words instead of 8-bit ones, so it needs four times the memory, and every calculation uses floating-point units that escalate the cost. This was evident from the fact that the 5x5 floating-point cascade design exceeded the available LUTs during implementation on the first board, that was sufficient for all other designs.

Figure 5-8 shows the hardware cost in LUTs compared to the image accuracy achieved, based on the Root Mean Squared Error (RMSE). Accuracy is significantly higher for floating-point implementations of filters with non-integer coefficients (Gaussian and Cascades) but at a noticeable hardware cost. All Sobel filter implementations achieve the same RMSE value and are affected by the architecture only in regards to resources spent.

6.2. Summary of findings and answers to research questions

This section summarizes the answers to the research questions defined in section 1.2.

RQ1. How does changing from floating-point arithmetic to integer fixed-point approximations affect image quality compared to savings in hardware resource usage?

In general, floating-point arithmetic provides accuracy in a wide range of values, but it demands significantly more hardware resources. Depending on the filter type and the value of the coefficients, the IEEE format can be redundant. In filters like Sobel edge detection where the kernel values are integers, the floating-point architecture does not offer better accuracy than the cheaper fixed-point datapath. They both produce the same result, identical to the reference. In filters like the Gaussian blur that has fractional coefficients, it is substantially more accurate. For the 3x3 kernel the RMSE against the reference drops from 1.39 in the integer design to 0.08 in the floating-point design, at the cost of roughly 3.4 times the LUTs and 18 additional DSP blocks.

RQ2. How do hardware resource consumption, maximum operating frequency ($F_{\max}$), and latency change when the kernel size increases from 3x3 to 5x5?

Increasing the kernel size from 3x3 to 5x5 raises every resource metric and lowers the maximum operating frequency for both arithmetic representations. For the floating-point Sobel filter, LUT usage grows from 7.4% to 23.9% of the device available slices and DSP usage from 14 to 66 blocks. For every filter case, the initial latency doubles when increasing kernel size, and the maximum operating frequency is decreased, by a greater value in the fixed-point designs, and a smaller one in the floating-point ones.

RQ3. How much do errors accumulate when connecting multiple filters in a sequence using fixed-point arithmetic, compared to a floating-point implementation?

In the integer cascade implementations, the truncation error of the first stage is passed to the second filter and amplified. Both the 3x3 and the 5x5 cases have small, but noticeable divergence from the reference, with RMSE values 1.19 and 0.87 respectively. The floating-point cascade maintains the value between the stages in full precision and consequently the result is nearly identical to the reference, with a maximum absolute difference of 1 and an RMSE below 0.01 for both kernel sizes. So, errors accumulate when connecting filters in a sequence, due to truncation in the integer arithmetic architectures, but not that intensely in the floating-point implementations.

RQ4. How do the achievable frame rate and the hardware area efficiency differ between the integer and floating-point datapaths, and how do they scale with kernel size?

Frame rate decreases with kernel scaling and pipeline complexity, from 378.1 fps in 3x3 Sobel integer to 299.8 fps in 5x5 Sobel integer and 293.2 in 3x3 Sobel floating-point. Area efficiency is lower for the floating-point implementations, since they use significantly more hardware resources. Also, between the same arithmetic representation, the increase of the kernel size decreases area efficiency by more than half, so it is lowest for the floating-point cascades.

6.3. Limitations

This study has several limitations that restrict the generalization of the findings.

The quality evaluation of the designs relies on a single 364x483 grayscale image. Although the image contains edges, textures and varying intensities, it cannot represent the full range of possible image characteristics. The hardware resource and timing results are largely independent of image characteristics, but image quality metrics can change significantly for images with specifications.

The evaluation is also limited to Sobel edge detection and Gaussian blur, two linear spatial filters with 3x3 and 5x5 kernels. Other common filtering operators that use second derivatives, or different coefficient bit-widths or non-linear neighborhood operations, were not implemented. Also, the scaling of the window to even larger values was not implemented. Those differences would influence a lot of factors such as memory, routing, timing, and area, so they remain to be explored in the future.

The floating-point datapath implements the sign, exponent and mantissa structure, but full compliance with the IEEE 754 standard was not evaluated. Special values, alternative rounding modes and all overflow or underflow cases were not part of the intended image processing application. The results consequently apply to the custom floating-point implementation used in this thesis and not necessarily to fully compliant IEEE-754 processing units.

Finally, the designs were evaluated through behavioral simulation, synthesis and implementation reports. They were not executed on a physical FPGA board. Maximum operating frequency, throughput and frame rate are estimations derived from Vivado and not actual measurements of a deployed system. Power consumption, energy per frame and external I/O behavior were not evaluated. The resource and timing results are also specific to the selected board, Vivado version and implementation settings.

6.4. Future work

There are multiple improvements to be made in this design. This section mentions some of the most important.

A future evaluation should include a larger image dataset with natural images and synthetic patterns such as constant fields, ramps, checkerboards, and saturation cases. Results should be reported as the mean, range, and worst case across the whole dataset, and not only one individual image. Also, new filters and filter combinations should be examined, while recording the error after every stage, to obtain a better understanding of the error propagation in multistage pipelines.

Another useful extension would be to compare the current integer and binary32 datapaths with fractional fixed-point formats and custom floating-point widths. This would allow the identification of the minimum word-length needed to achieve a certain precision. The design should be implemented on a physical board to obtain real values about performance, timing and energy consumption. This can be integrated with a streaming interface to fully simulate a real camera-to-display pipeline.

6.5. Conclusions

This thesis demonstrates that numerical precision and representation should be selected according to the requirements of the filter. For the tested Sobel kernels, integer arithmetic produced the same output as floating point at substantially lower hardware cost. For Gaussian and cascaded filters, maintaining intermediate precision improves output accuracy but requires more hardware resources and in the largest case, a larger FPGA.

Fixed-point arithmetic is preferable when the approximation error remains within the required tolerance. Floating-point representations are suitable for designs with fractional coefficients or with multistage error propagation that affects output quality. These conclusions are valid for the implemented designs and test image. To be able to generalize these results, various tests and changes are needed.

References

- [1] Nikolić, G. S., Dimitrijević, B. R., Nikolić, T. R., & Stojčev, M. K. (2022). A survey of three types of processing units: CPU, GPU and TPU. In *2022 57th International Scientific Conference on Information, Communication and Energy Systems and Technologies (ICEST)* (pp. 1–6). IEEE. <https://doi.org/10.1109/ICEST55168.2022.9828625>
- [2] Qasaimeh, M., Denolf, K., Lo, J., Vissers, K., Zambreno, J., & Jones, P. H. (2019). Comparing energy efficiency of CPU, GPU and FPGA implementations for vision kernels. In *2019 IEEE International Conference on Embedded Software and Systems (ICEST)* (pp. 1–8). IEEE. <https://doi.org/10.1109/ICEST.2019.8782524>
- [3] Nickolls, J., & Dally, W. J. (2010). The GPU computing era. *IEEE Micro*, 30(2), 56–69. <https://doi.org/10.1109/MM.2010.41>
- [4] Asano, S., Maruyama, T., & Yamaguchi, Y. (2009). Performance comparison of FPGA, GPU and CPU in image processing. In *2009 International Conference on Field Programmable Logic and Applications (FPL)* (pp. 126–131). IEEE. <https://doi.org/10.1109/FPL.2009.5272532>
- [5] Ruiz-Rosero, J., Ramirez-Gonzalez, G., & Khanna, R. (2019). Field programmable gate array applications—A scientometric review. *Computation*, 7(4), 63. <https://doi.org/10.3390/computation7040063>
- [6] Boutros, A., & Betz, V. (2021). FPGA architecture: Principles and progression. *IEEE Circuits and Systems Magazine*, 21(2), 4–29. <https://doi.org/10.1109/MCAS.2021.3071607>
- [7] Yang, H., Yu, L., Zhang, J., & Sun, J. (2014). Review of advanced FPGA architectures and technologies. *Journal of Electronics (China)*, 31(5), 371–393. <https://doi.org/10.1007/s11767-014-4090-x>
- [8] D. G. Bailey, *Design for Embedded Image Processing on FPGAs*, 2nd ed. Chichester, UK: Wiley-IEEE Press, 2023. ISBN 978-1-119-81979-0. DOI: 10.1002/9781119819820.
- [9] AMD. (2026). *Vivado Design Suite user guide: Using constraints* (UG903 v2026.1). AMD.
- [10] AMD. (2026). *UltraFast design methodology guide for FPGAs and SoCs* (UG949 v2026.1). AMD.
- [11] Kastner, R., Matai, J., & Neuendorffer, S. (2018). *Parallel programming for FPGAs: The HLS book*. arXiv. <https://doi.org/10.48550/arXiv.1805.03648>
- [12] Shi, R., Wong, J. S. J., & So, H. K.-H. (2019). High-Throughput Line Buffer Microarchitecture for Arbitrary Sized Streaming Image Processing. *Journal of Imaging*, 5(3), 34. <https://doi.org/10.3390/jimaging5030034>
- [13] Yates, R. (2001). *Fixed-point arithmetic: An introduction*. Digital Sound Labs.
- [14] Ziaullah, M., & Munaff, A. (2015). Design and implementation of floating point ALU with parity generator using Verilog HDL. *IOSR Journal of VLSI and Signal Processing (IOSR-JVSP)*, 5(5), 54–59. <https://doi.org/10.9790/4200-05515459>
- [15] Voronenko, Y., & Püschel, M. (2007). Multiplierless multiple constant multiplication. *ACM Transactions on Algorithms*, 3(2), 1–39. <https://doi.org/10.1145/1240233.1240242>
- [16] Ene, T.-D., & Stine, J. E. (2021). A comprehensive exploration of the parallel prefix adder tree space. In *2021 IEEE 39th International Conference on Computer Design (ICCD)* (pp. 125–129). IEEE. <https://doi.org/10.1109/ICCD53106.2021.00030>
- [17] Ravichandran, S., Su, H.-K., Kuo, W.-K., Dhanasekaran, D., Mahalingam, M., & Yang, J.-P. (2025). Parallel processing of Sobel edge detection on FPGA: Enhancing real-time image analysis. *Sensors*, 25(12), Article 3649. <https://doi.org/10.3390/s25123649>
- [18] Cabello, F., León, J., Iano, Y., & Arthur, R. (2015). Implementation of a fixed-point 2D Gaussian filter for image processing based on FPGA. In *2015 Signal Processing: Algorithms, Architectures, Arrangements, and Applications (SPA)* (pp. 28–33). IEEE. <https://doi.org/10.1109/SPA.2015.7365108>
- [19] Al-Najjar, Y. A. Y., & Soong, D. C. (2012). Comparison of image quality assessment: PSNR, HVS, SSIM, UIQI. *International Journal of Scientific & Engineering Research*, 3(8), 1–5.

- [20] Coussy, P., Gajski, D. D., Meredith, M., & Takach, A. (2009). An introduction to high-level synthesis. *IEEE Design & Test of Computers*, 26(4), 8-17. <https://doi.org/10.1109/MDT.2009.69>
- [21] Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004). Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4), 600–612. <https://doi.org/10.1109/TIP.2003.819861>
- [22] Bailey, D. G. (2024). Design for embedded image processing on FPGAs (2nd ed.). Wiley-IEEE Press. <https://doi.org/10.1002/9781119819820>
- [23] Al-Dujaili, A., & Fahmy, S.A. (2017). High Throughput 2D Spatial Image Filters on FPGAs. ArXiv, 1710.05154. <https://doi.org/10.48550/arXiv.1710.05154>
- [24] Cooke, P., Fowers, J., Brown, G., & Stitt, G. (2015). A tradeoff analysis of FPGAs, GPUs, and multicores for sliding-window applications. *ACM Transactions on Reconfigurable Technology and Systems*, 8(1), 1-24. <https://doi.org/10.1145/2659000>
- [25] Sreenivasulu, M., & Meenpal, T. (2019). Efficient hardware implementation of 2D convolution on FPGA for image processing application. In 2019 IEEE International Conference on Electrical, Computer and Communication Technologies (ICECCT) (pp. 1-5). IEEE. <https://doi.org/10.1109/ICECCT.2019.8869347>
- [26] Kho, D. C. K., Fauzi, M. F. A., & Lim, S. L. (2020). Hardware parallel processing of 3x3-pixel image kernels. In 2020 IEEE Region 10 Conference (TENCON) (pp. 1272-1276). IEEE. <https://doi.org/10.1109/TENCON50793.2020.9293914>
- [27] Saranya, C., Kumar, P., Boomika, R., & Harish, G.S. (2025). Implementation of a Line Buffer in an Adaptive Gaussian Bilateral Filter, in 2025 5th International Conference on Pervasive Computing and Social Networking (ICPCSN), Salem, India, pp. 324-329. IEEE. <https://doi.org/10.1109/ICPCSN65854.2025.11034915>.
- [28] Bailey, D. G., & Johnston, C. T. (2010). Algorithm transformation for FPGA implementation. In 2010 Fifth IEEE International Symposium on Electronic Design, Test & Applications (pp. 77–81). IEEE. <https://doi.org/10.1109/DELTA.2010.17>
- [29] Singh, S., Saurav, S., Saini, R., Saini, A.K. Shekhar, C., & Vohra, A. (2014). Comprehensive Review and Comparative Analysis of Hardware Architectures for Sobel Edge Detector, International Scholarly Research Notices, Article 857912. <https://doi.org/10.1155/2014/857912>
- [30] Chaple, G., & Daruwala, R. D. (2014). Design of Sobel operator based image edge detection algorithm on FPGA. In 2014 International Conference on Communication and Signal Processing, Melmaruvathur, India, pp. 788-792. IEEE. <https://doi.org/10.1109/ICCSP.2014.6949951>
- [31] Chaple, G. N., Daruwala, R. D., & Gofane, M. S. (2015). Comparisons of Robert, Prewitt, Sobel operator based edge detection methods for real time uses on FPGA. In 2015 International Conference on Technologies for Sustainable Development (ICTSD), Mumbai, India, pp. 1-4 . IEEE. <https://doi.org/10.1109/ICTSD.2015.7095920>
- [32] Kwon, Y. H., & Jeon, J. W. (2020). Comparison of FPGA implemented Sobel edge detector and Canny edge detector. In 2020 IEEE International Conference on Consumer Electronics - Asia (ICCE-Asia), pp. 1-2. IEEE. <https://doi.org/10.1109/ICCE-Asia49877.2020.9277425>
- [33] Jiang, J., Liu, C., & Ling, S. (2018). An FPGA implementation for real-time edge detection. *Journal of Real-Time Image Processing*, 15, 787-797. <https://doi.org/10.1007/s11554-015-0521-7>
- [34] Kashyap, S., Bhandari, A. K., & Giri, P. (2024). Low resource FPGA implementation based efficient image edge detector architecture. *Multimedia Tools and Applications*, 83, 25595-25615. <https://doi.org/10.1007/s11042-023-16472-w>
- [35] Ivashko, A., Zuev , A., Karaman, D., & Moškon, M. (2024). FPGA-based implementation of a Gaussian smoothing filter with powers-of-two coefficients. (2024). *Advanced Information Systems*. <https://doi.org/10.20998/2522-9052.2024.2.05>
- [36] Spagnolo, F., Corsonello, P., Frustaci, F., & Perri, S. (2023). Design of approximate bilateral filters for image denoising on FPGAs. *IEEE Access*, 11, 1990-2000. <https://doi.org/10.1109/ACCESS.2022.3233921>

- [37] ISO/IEC/IEEE International Standard - Floating-point arithmetic, in ISO/IEC 60559:2020(E) IEEE Std 754-2019 , pp.1-86, 8 May 2020, <https://doi.org/10.1109/IEEESTD.2020.9091348>
- [38] Kim, S., & Sung, W. (1996). Fixed-point error analysis and wordlength optimization of 8x8 IDCT architectures. in VLSI Signal Processing, IX, San Francisco, CA, USA, 1996, pp. 398-407. <https://doi.org/10.1109/VLSISP.1996.558372>
- [39] Constantinides, G. A., Cheung, P. Y. K., & Luk, W. (2003). Wordlength optimization for linear digital signal processing. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 22(10), 1432-1442. <https://doi.org/10.1109/TCAD.2003.818119>
- [40] Menard, D., Rocher, R., & Sentieys, O. (2008). Analytical fixed-point accuracy evaluation in linear time-invariant systems. IEEE Transactions on Circuits and Systems I: Regular Papers, 55(10), 3197-3208. <https://doi.org/10.1109/TCSI.2008.923279>
- [41] Nane, R., Sima, V.-M., Pilato, C., Choi, J., Fort, B., Canis, A., Chen, Y. T., Hsiao, H., Brown, S., Ferrandi, F., Anderson, J., & Bertels, K. (2016). A survey and evaluation of FPGA high-level synthesis tools. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 35(10), 1591-1604. <https://doi.org/10.1109/TCAD.2015.2513673>
- [42] Lahti, S., Sjövall, P., Vanne, J., & Hämäläinen, T. D. (2019). Are we there yet? A study on the state of high-level synthesis. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 38(5), 898-911. <https://doi.org/10.1109/TCAD.2018.2834439>
- [43] Gurel, M. (2016). A comparative study between RTL and HLS for image processing applications with FPGAs [Master's thesis, University of California San Diego]. eScholarship. <https://escholarship.org/uc/item/9vx1s37b>
- [44] Guo, L., & Wu, S. (2023). FPGA implementation of a real-time edge detection system based on an improved Canny algorithm. *Applied Sciences*, 13(2), Article 870. <https://doi.org/10.3390/app13020870>
- [45] Menard, D., Herve, N., Sentieys, O. and Nguyen, H.-N. (2012). High-level synthesis under fixed-point accuracy constraint. *Journal of Electrical and Computer Engineering*, Article ID 906350. <https://doi.org/10.1155/2012/906350>

APPENDIX A – ACRONYMS AND ABBREVIATIONS

Acronym	Explanation
1D	One-Dimensional
2D	Two-Dimensional
ALU	Arithmetic Logical Unit
BRAM	Block Random Access Memory
CPU	Central Processing Unit
DSP	Digital Signal Processing block (arithmetic slice in an FPGA)
FF	Flip-Flop
FIFO	First In First Out
FPGA	Field Programmable Gate Array
FPS	Frames Per Second
GPU	Graphics Processing Unit
HDL	Hardware Description Language
HLS	High-Level Synthesis
I/O	Input/Output
IEEE	Institute of Electrical and Electronics Engineers
LUT	Look-Up Table
MAC	Multiply-Accumulate
MAD	Maximum Absolute Difference
MSB	Most Significant Bit
MSE	Mean Squared Error
NumPy	Numerical Python (Python numerical computing library)
OpenCV	Open-Source Computer Vision Library
PSNR	Peak Signal-to-Noise Ratio
RAM	Random Access Memory

Acronym	Explanation
RGB	Red, Green, Blue (color model)
RMSE	Root Mean Squared Error
RQ	Research Question
RTL	Register Transfer Level
SIMD	Single Instruction, Multiple Data
SIMT	Single Instruction, Multiple Threads
SSIM	Structural Similarity Index Measure
WNS	Worst Negative Slack
VHDL	VHSIC Hardware Description Language
VHSIC	Very High-Speed Integrated Circuit

APPENDIX B – SYMBOLS

Symbol	Explanation
T_{clk}	Clock period
F	Clock frequency
F_{max}	Maximum operating frequency
FPS	Frames per second
//	Initiation Interval
K	Kernel (window) size, for example $K = 3$ for a 3×3 window
L	Latency
T	Data throughput in pixels per second
T_{min}	Minimum operating period
T_{target}	Target clock period used as the timing constraint
W, H	Image width and height in pixels
WNS	Worst Negative Slack